

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT
THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO TIỂU LUẬN
NGÔN NGỮ LẬP TRÌNH PYTHON

TÊN ĐỀ TÀI

**HỆ THỐNG NHẬN DIỆN NGÔN NGỮ KÝ HIỆU ASL
BẰNG MACHINE LEARNING VÀ MEDIAPIPE**

Mã lớp học: IPPA233277

Thông tin thành viên:

23110327	Huỳnh Ngọc Thắng	241IPPA233277_05
23110225	Nguyễn Lâm Huy	241IPPA233277_05
23110308	Nguyễn Thành Tài	241IPPA233277_05
23110319	Hồ Minh Tiến Thành	241IPPA233277_05

Giảng viên: TS. Phan Thị Thể

Thành phố Hồ Chí Minh, tháng 12 năm 2024

**BẢNG PHÂN CÔNG CÔNG VIỆC VÀ ĐÁNH GIÁ HOÀN THÀNH CỦA CÁC
THÀNH VIÊN TRONG NHÓM**

STT	Họ và tên	Phân công công việc	Tỷ lệ hoàn thành
1	Huỳnh Ngọc Thắng	- Tìm tài liệu và các nguồn liên quan đến đề tài cũng như là tìm hiểu về machine learning, đặc biệt là mô hình học máy Random Forest Classifier. - Thực hiện tìm kiếm dữ liệu, xử lý dữ liệu và rèn luyện ra các model máy học để có thể ứng dụng vào đề tài. - Tìm hiểu về cách nâng cao độ chính xác và điều chỉnh dữ liệu đầu vào để cải thiện model. - Viết, tổng hợp và điều chỉnh báo cáo của đề tài và hỗ trợ thiết kế slide. - Thực hiện công việc thuyết trình về đề tài.	100%
2	Nguyễn Lâm Huy	- Ứng dụng model đã được rèn luyện để xây dựng ra chức năng nhận diện ký tự và in các ký tự đó ra màn hình. - Cài đặt tính năng âm thanh để có thể phát ra giọng nói từ các ký tự đã xuất trên màn hình. - Viết báo cáo và hỗ trợ thiết kế slide.	100%

3	Nguyễn Thành Tài	- Ứng dụng model đã được rèn luyện để phát triển ra trò chơi với ngôn ngữ ký hiệu - Thực hiện cài đặt các hàm cũng như âm thanh trong trò chơi để hệ thống hoạt động một cách hiệu quả. - Viết báo cáo và thiết kế slide.	100%
4	Hồ Minh Tiến Thành	- Từ quá trình demo của các chức năng, thiết kế nên giao diện hệ thống và menu (các nút, thông báo, event,...) của toàn bộ tính năng. Xử lý các tính năng đóng mở giao diện. - Thiết kế ra chức năng, giao diện học ngôn ngữ kí hiệu “Learn the ASL”. - Hỗ trợ các bạn khác trong việc tìm thông tin và phát triển các tính năng. - Viết báo cáo và thiết kế slide.	100%

MỤC LỤC

DANH MỤC BẢNG BIỂU, HÌNH ẢNH	1
TÓM TẮT ĐỀ TÀI.....	2
MỞ ĐẦU.....	3
1. Lý do thực hiện đề tài.....	3
2. Ý nghĩa và mục tiêu của dự án.....	4
3. Phạm vi thực hiện.....	4
4. Bố cục của bản báo cáo.....	5
NỘI DUNG.....	6
CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VỀ NGÔN NGỮ KÝ HIỆU VÀ MÔ HÌNH HỌC MÁY	6
1.1. Tổng quan về ngôn ngữ ký hiệu – ASL	6
1.2. Random Forest Classifier và các thuật toán máy học nhận diện	6
1.2.1. Các khái niệm cơ bản.....	6
1.2.2. Phân loại các mô hình máy học phổ biến	8
1.2.3. Giới thiệu về Scikit-learn	9
1.3. Mediapipe và ứng dụng trong nhận diện bàn tay	10
1.3.1. Giới thiệu về mediapipe	10
1.3.2. Triển khai nhận diện tay bằng mediapipe.....	11
CHƯƠNG 2: THIẾT KẾ HỆ THỐNG VÀ CÁC THƯ VIỆN BỔ SUNG.....	13
2.1. Thiết kế hệ thống	13
2.1.1. Kiến trúc tổng quan	13
2.1.2. Thiết kế mô hình học máy Random Forest.....	13
2.1.3. Thiết kế giao diện.....	14
2.2. Công cụ và thư viện khác.....	17
2.2.1. OpenCV	18
2.2.2. Pandas và Pickle.....	19
2.2.3. Google.generativeai.....	20
2.2.4. gTTS.....	20
CHƯƠNG 3: TRIỂN KHAI, CẢI TIẾN VÀ THỰC NGHIỆM	22
3.1. Dữ liệu và xử lý dữ liệu	22
3.1.1. Thu thập dữ liệu ASL Dataset.....	22
3.1.2. Xử lý dữ liệu.....	24

3.1.3. Lưu trữ dữ liệu.....	25
3.2. Training mô hình.....	26
3.2.1. Các tham số đầu vào n_estimators, max_depth, random_state.....	26
3.2.2. Quá trình rèn luyện mô hình.....	27
3.2.3. Độ chính xác lý thuyết của mô hình	28
3.3. Các tính năng cải tiến của đề tài.....	29
3.3.1. Tính năng Text-to-Speech	29
3.3.2. Trò chơi với nhận diện ngôn ngữ ký hiệu ASL.....	31
3.3.3. Tính năng “Learn the ASL”.....	33
CHƯƠNG 4: KẾT QUẢ THỬ NGHIỆM VÀ ĐÁNH GIÁ TỔNG QUAN	34
4.1. Kết quả	34
4.1.1. Độ chính xác thực tế.....	34
4.1.2. Kết quả các tính năng.....	35
4.2. Đánh giá tổng quan	36
KẾT LUẬN VÀ KHUYẾN NGHỊ.....	37
1. Kết luận.....	37
2. Hướng phát triển trong tương lai	38
TÀI LIỆU THAM KHẢO.....	40

DANH MỤC BẢNG BIỂU, HÌNH ẢNH

Bảng 1. Bảng biểu thị độ chính xác lý thuyết dựa trên các tham số đầu vào	29
Hình 1. Hình về ngôn ngữ ký hiệu - ASL.....	6
Hình 2. Hình về quy trình làm việc với machine learning.....	8
Hình 3. Hình về các loại nhận dạng phổ biến của mediapipe.....	11
Hình 4. Hình mô phỏng cấu trúc xương tay.....	12
Hình 5. Hình ví dụ về mô hình Random Forest.....	14
Hình 6. Hình giao diện menu.....	16
Hình 7. Hình giao diện ASL to Voice.....	16
Hình 8. Hình giao diện ASL Sign Game	17
Hình 9. Hình giao diện Learn The ASL	17
Hình 10. Hình ảnh mô phỏng về quá trình thu thập dữ liệu qua OpenCV	22
Hình 11. Hình ảnh mô phỏng thư mục lưu trữ dữ liệu	23
Hình 12. Sơ đồ cho quá trình thu thập hình ảnh và đánh nhãn.....	23
Hình 13. Hình ảnh về quá trình xác định tọa độ cho cấu trúc tay.....	25
Hình 14. Hình ảnh về phương thức lưu trữ dữ liệu sau khi xử lý	26
Hình 15. Hình ảnh hưởng của n_estimators đến độ chính xác lý thuyết.....	26
Hình 16. Hình ảnh hưởng của max_depth đến độ chính xác lý thuyết.....	27
Hình 17. Hình ảnh về quá trình rèn luyện mô hình và lưu mô hình.....	28
Hình 18. Hình ảnh về tính năng đoán ký tự và xuất ra giọng nói	30
Hình 19. Hình ảnh về tính năng trò chơi	31
Hình 20. Hình ảnh về các hàm được sử dụng trong trò chơi.....	32
Hình 21. Hình tính năng Learn hoạt động song song với trò chơi.....	33
Hình 22. Hình so sánh độ chính xác giữa môi trường bị ngược sáng và không bị ngược sáng	35

TÓM TẮT ĐỀ TÀI

Đề tài nghiên cứu về nhận diện ngôn ngữ ký hiệu Mỹ (ASL) nhằm phát triển một hệ thống có khả năng nhận diện và chuyển đổi các cử chỉ ngôn ngữ ký hiệu thành dạng văn bản hoặc âm thanh. Mục đích chính của dự án là tạo ra một cầu nối giao tiếp giữa người khiếm thính và những người không biết ngôn ngữ ký hiệu, từ đó thúc đẩy sự hòa nhập và tương tác xã hội.

Hệ thống nhận diện ASL được thiết kế với quy trình rõ ràng và mạch lạc, bao gồm các bước từ thu thập dữ liệu, tiền xử lý, huấn luyện mô hình, đến triển khai và kiểm thử. Đầu tiên, hệ thống thu thập một bộ dữ liệu phong phú gồm các hình ảnh và video chứa các cử chỉ ngôn ngữ ký hiệu từ nhiều nguồn khác nhau. Sau đó, dữ liệu này được tiền xử lý để cải thiện chất lượng như cắt hay chuẩn hóa kích thước hình ảnh.

Trong giai đoạn huấn luyện mô hình, hệ thống sử dụng mô hình Random Forest để nhận diện các ký hiệu ASL. Random Forest là một phương pháp học máy tổng hợp mạnh mẽ, hoạt động bằng cách xây dựng nhiều cây quyết định và kết hợp các cây này để đưa ra dự đoán cuối cùng. Bộ dữ liệu đã tiền xử lý được chia thành tập huấn luyện và tập kiểm tra để đánh giá hiệu suất của mô hình. Các chỉ số như độ chính xác, độ nhạy và độ đặc hiệu được sử dụng để đo lường hiệu quả của mô hình.

Sau khi huấn luyện, hệ thống được tích hợp với một giao diện người dùng thân thiện và dễ sử dụng, cho phép người dùng thực hiện các cử chỉ tay chứa các ký hiệu ASL và nhận được kết quả nhận diện. Hệ thống cũng hỗ trợ tính năng chuyển đổi văn bản thành giọng nói (Text-to-Speech - TTS) để người dùng không biết ngôn ngữ ký hiệu có thể hiểu được thông tin.

Trong quá trình thử nghiệm, hệ thống đã được kiểm tra qua nhiều tính năng như chuyển đổi văn bản thành câu hoàn chỉnh, nhận diện và chuyển văn bản thành giọng nói, chơi game nhận diện cử chỉ tay (ASL Game), điều khiển âm thanh trong trò chơi và hiển thị thời gian và điểm số. Các thử nghiệm cho thấy hệ thống hoạt động mượt mà và đáp ứng tốt yêu cầu của người dùng.

MỞ ĐẦU

1. Lý do thực hiện đề tài

Việc lựa chọn đề tài nhận diện ngôn ngữ ký hiệu Mỹ (ASL) dựa trên nhiều lý do mạnh mẽ và quan trọng, thể hiện sự kết hợp giữa nhu cầu xã hội, tiềm năng công nghệ và khả năng ứng dụng thực tiễn. Hàng triệu người trên khắp thế giới sử dụng ngôn ngữ ký hiệu như một phương tiện giao tiếp chính. Nhận diện ngôn ngữ ký hiệu không chỉ giúp cải thiện chất lượng cuộc sống của những người khiếm thính mà còn thúc đẩy sự hòa nhập và tương tác xã hội. Việc phát triển một hệ thống nhận diện ASL có thể giúp tạo cầu nối giao tiếp giữa người khiếm thính và những người không biết ngôn ngữ ký hiệu, từ đó thúc đẩy sự hiểu biết và thông cảm lẫn nhau trong cộng đồng.

Công nghệ trí tuệ nhân tạo và học máy đang phát triển mạnh mẽ, cung cấp nhiều công cụ và phương pháp để giải quyết các bài toán phức tạp. Những tiến bộ mới nhất trong lĩnh vực học máy và xử lý ảnh có thể được tận dụng để phát triển một hệ thống nhận diện ASL hiệu quả và chính xác. Dự án nhận diện ngôn ngữ ký hiệu có tiềm năng ứng dụng rộng rãi trong nhiều lĩnh vực, từ giáo dục, y tế đến dịch vụ khách hàng. Ví dụ, trong giáo dục, hệ thống có thể hỗ trợ việc giảng dạy và học tập ngôn ngữ ký hiệu, giúp học sinh và giáo viên giao tiếp dễ dàng hơn. Trong lĩnh vực y tế, hệ thống có thể hỗ trợ các y bác sĩ giao tiếp với bệnh nhân khiếm thính, nâng cao chất lượng dịch vụ chăm sóc sức khỏe. Ngoài ra, trong các dịch vụ khách hàng, hệ thống có thể được tích hợp để hỗ trợ giao tiếp với người khiếm thính, cải thiện trải nghiệm và sự hài lòng của khách hàng.

Dự án này mang lại nhiều thách thức kỹ thuật, từ việc xử lý và phân tích hình ảnh đến việc thiết kế và huấn luyện mô hình học máy. Tuy nhiên, chính những thách thức này cung cấp cơ hội tuyệt vời để học hỏi và phát triển kỹ năng trong các lĩnh vực như lập trình, học máy, và xử lý ảnh. Qua quá trình thực hiện, dự án sẽ cung cấp kiến thức và kinh nghiệm, đồng thời tạo ra một sản phẩm có giá trị thực tiễn cao.

Cuối cùng, việc phát triển một hệ thống nhận diện ngôn ngữ ký hiệu không chỉ mang lại lợi ích cá nhân mà còn có thể đóng góp tích cực cho cộng đồng. Sản phẩm hoàn thiện có thể được chia sẻ và sử dụng rộng rãi, giúp nhiều người hơn có khả năng tiếp cận và sử dụng ngôn ngữ ký hiệu, từ đó tạo ra một xã hội toàn diện và hòa nhập hơn. Tóm lại,

việc chọn đề tài nhận diện ngôn ngữ ký hiệu Mỹ không chỉ thể hiện ý nghĩa nhân văn và xã hội mà còn đem lại nhiều cơ hội học tập và phát triển công nghệ. Đây là một lựa chọn đúng đắn và tiềm năng, với những giá trị thực tiễn và lợi ích sâu rộng.

2. Ý nghĩa và mục tiêu của dự án

Đề tài nghiên cứu về nhận diện ngôn ngữ ký hiệu Mỹ (ASL) mang lại nhiều ý nghĩa quan trọng trên nhiều phương diện, từ khoa học công nghệ, xã hội đến nhân văn.

Trước hết, từ góc độ khoa học công nghệ, việc phát triển một hệ thống nhận diện ASL thể hiện sự tiến bộ và ứng dụng thực tiễn của các công nghệ hiện đại như trí tuệ nhân tạo, học máy, và xử lý ảnh. Dự án này không chỉ giúp làm sáng tỏ khả năng của các mô hình học máy trong việc giải quyết các vấn đề phức tạp mà còn mở ra nhiều hướng nghiên cứu mới trong lĩnh vực nhận diện hình ảnh và giao tiếp không lời.

Đề tài nhận diện ngôn ngữ ký hiệu Mỹ mang trong mình giá trị đa chiều, bao gồm cả khoa học công nghệ, xã hội và nhân văn. Trước hết, về mặt khoa học, đề tài này là minh chứng cho khả năng ứng dụng tiên tiến của trí tuệ nhân tạo, học máy và xử lý ảnh trong việc giải quyết các thách thức giao tiếp phi ngôn ngữ. Thông qua việc nghiên cứu và triển khai, dự án không chỉ khẳng định vai trò của công nghệ trong việc hỗ trợ cuộc sống con người mà còn mở ra những hướng nghiên cứu mới, nâng cao hiệu quả trong lĩnh vực nhận diện hình ảnh.

Về mặt xã hội, hệ thống nhận diện ASL góp phần phá bỏ rào cản giao tiếp giữa người khiếm thính và cộng đồng. Điều này không chỉ tạo ra môi trường giao tiếp thuận lợi hơn trong học tập, làm việc và các hoạt động hàng ngày, mà còn thúc đẩy sự hòa nhập và gắn kết xã hội, xây dựng một cộng đồng biết lắng nghe và chia sẻ.

Ngoài ra, ý nghĩa nhân văn của đề tài cũng được thể hiện rõ nét khi nó mang đến sự hỗ trợ thiết thực cho nhóm đối tượng dễ bị tổn thương. Nỗ lực phát triển hệ thống này không chỉ giúp giảm bớt khó khăn mà người khiếm thính gặp phải, mà còn nâng cao nhận thức xã hội về quyền lợi và tiềm năng của họ. Điều này khuyến khích một tinh thần cộng đồng đồng cảm và sẵn sàng giúp đỡ lẫn nhau.

Tóm lại, dự án không chỉ tạo ra những giá trị khoa học và công nghệ mà còn mang lại lợi ích thiết thực cho cộng đồng và khẳng định tinh thần nhân văn sâu sắc.

3. Phạm vi thực hiện

Trọng tâm của đề tài là ngôn ngữ ký hiệu Mỹ (ASL), một trong những ngôn ngữ ký hiệu phổ biến và được sử dụng rộng rãi tại Mỹ và nhiều quốc gia khác. ASL có hệ thống ký hiệu phong phú và hoàn chỉnh, bao gồm các cử chỉ tay, biểu cảm khuôn mặt và chuyển động cơ thể. Nghiên cứu này sẽ tập trung vào việc nhận diện các ký hiệu và cử chỉ trong ASL để đảm bảo tính chính xác và hiệu quả của hệ thống.

Phạm vi nghiên cứu bao gồm việc nhận diện các chữ cái trong bảng chữ cái ASL cũng như các ký hiệu đơn giản và phức tạp. Việc nhận diện chữ cái là bước đầu tiên và cơ bản để xây dựng hệ thống nhận diện ngôn ngữ ký hiệu. Sau đó, hệ thống sẽ mở rộng để nhận diện các ký hiệu phức tạp hơn. Điều này đòi hỏi hệ thống phải có khả năng phân tích và nhận diện chính xác các chuyển động và biểu cảm khác nhau.

Mặc dù trọng tâm là ngôn ngữ ký hiệu Mỹ, phạm vi ứng dụng của hệ thống này có thể mở rộng ra toàn cầu. Các công nghệ được phát triển trong nghiên cứu này có thể được điều chỉnh và áp dụng cho các ngôn ngữ ký hiệu khác trên thế giới. Việc này không chỉ giúp mở rộng phạm vi ứng dụng của hệ thống mà còn góp phần thúc đẩy sự hòa nhập và tương tác giữa các cộng đồng ngôn ngữ ký hiệu khác nhau trên thế giới.

4. Bố cục của bản báo cáo

Ngoài phần mở đầu, kết luận, danh mục hình ảnh và tài liệu tham khảo thì phần nội dung được chia làm bốn chương chính:

Chương 1: Cơ sở lý thuyết về ngôn ngữ ký hiệu và mô hình học máy

Chương 2: Thiết kế hệ thống và các thư viện bổ sung

Chương 3: Triển khai, cải tiến và thực nghiệm

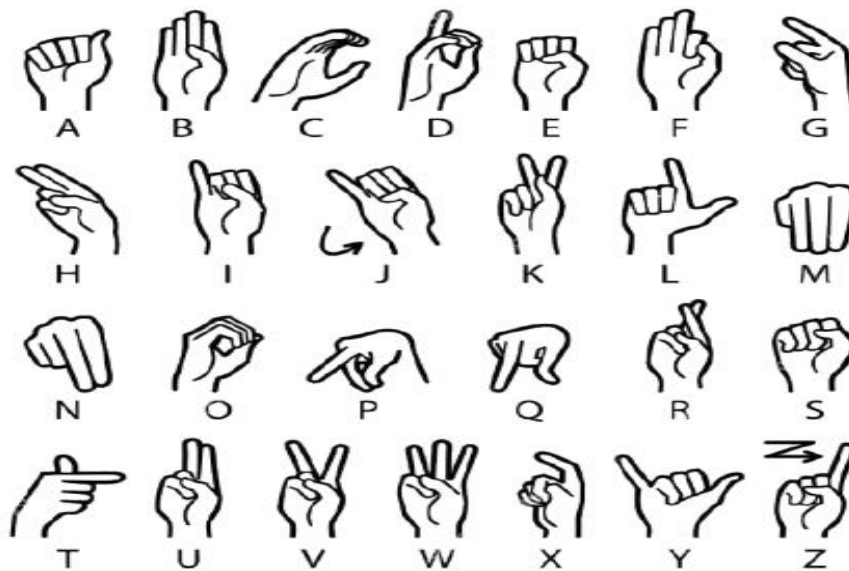
Chương 4: Kết quả thử nghiệm và đánh giá tổng quan

NỘI DUNG

CHƯƠNG 1: CƠ SỞ LÝ THUYẾT VỀ NGÔN NGỮ KÝ HIỆU VÀ MÔ HÌNH HỌC MÁY

1.1. Tổng quan về ngôn ngữ ký hiệu – ASL

Ngôn ngữ ký hiệu Mỹ (ASL - American Sign Language) là một hệ thống ngôn ngữ sử dụng các động tác tay, hình dáng, và biểu cảm khuôn mặt để truyền tải thông tin. ASL được sử dụng chủ yếu bởi cộng đồng người khiếm thính tại Mỹ và Canada. Ngôn ngữ này không chỉ đơn giản là một sự thay thế cho ngôn ngữ nói mà còn có cấu trúc ngữ pháp riêng biệt, độc lập và có khả năng diễn đạt nhiều khái niệm phức tạp. Không như nhiều người nghĩ, ASL không phụ thuộc vào ngôn ngữ nói của một quốc gia cụ thể mà tự nó có một hệ thống ngôn ngữ riêng biệt.



Hình 1. Hình về ngôn ngữ ký hiệu - ASL

1.2. Random Forest Classifier và các thuật toán máy học nhận diện

1.2.1. Các khái niệm cơ bản

Machine learning (ML) hay máy học là một nhánh của trí tuệ nhân tạo (AI), nó là một lĩnh vực nghiên cứu cho phép máy tính có khả năng cải thiện chính bản thân chúng dựa trên dữ liệu mẫu (training data) hoặc dựa vào kinh nghiệm (những gì đã được học).

Machine learning có thể tự dự đoán hoặc đưa ra quyết định mà không cần được lập trình cụ thể. Bài toán machine learning thường được chia làm hai loại là dự đoán (prediction) và phân loại (classification). Các bài toán dự đoán như dự đoán giá nhà, giá xe... Các bài toán phân loại như nhận diện chữ viết tay, nhận diện đồ vật...

Dataset (còn gọi là data corpus hay data stock): là tập dữ liệu ở dạng nguyên thủy chưa qua xử lý mà bạn đã thu thập được ở bước data collection (thu tập dữ liệu).

Model: là các mô hình được dùng để rèn luyện - training trên một lượng dữ liệu đầu vào theo thuật toán của mô hình đó. Sau đó mô hình có thể dự đoán hoặc đưa ra các quyết định dựa trên những gì chúng đã được học.

Machine learning workflow có thể được hiểu là quy trình để làm việc với machine learning. Cụ thể từng bước trong machine learning workflow như sau như sau:

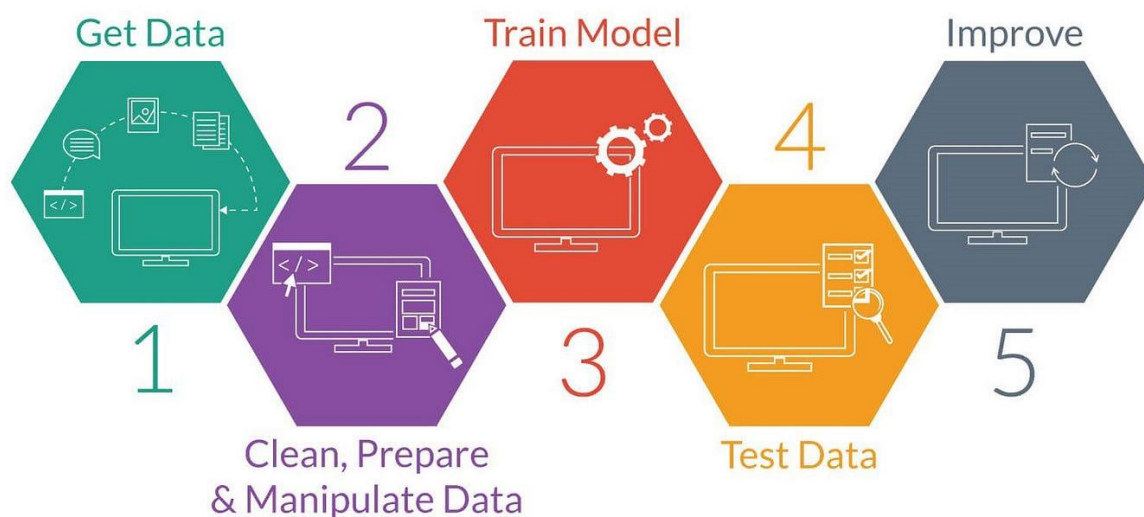
Thứ nhất, Data collection – thu thập dữ liệu: Bước đầu là để máy tính có thể học được thì cần có một bộ dữ liệu (dataset), có thể tự thu thập hoặc lấy các bộ dữ liệu đã được công bố trước đó. Các nguồn dữ liệu phải là nguồn chính xác và được xác định rõ ràng, có như vậy dữ liệu mới chính xác và máy có thể học một cách đúng đắn và đạt hiệu quả cao hơn.

Thứ hai, Preprocessing – tiền xử lý: Dùng để chuẩn hóa dữ liệu, loại bỏ các thuộc tính không cần thiết và gán nhãn dữ liệu, mã hóa một số đặc trưng, trích xuất đặc trưng, rút gọn dữ liệu nhưng vẫn đảm bảo kết quả... Bước này tốn thời gian nhất tỉ lệ thuận với số lượng dữ liệu đầu vào. Bước 1 và 2 thường chiếm hơn 70% tổng thời gian thực hiện trong quy trình làm việc với học máy.

Thứ ba, Training model – huấn luyện mô hình: Đây là bước để huấn luyện cho mô hình hay chính là cho nó học trên dữ liệu đã thu thập và xử lý ở hai bước đầu.

Thứ tư, Evaluating model – đánh giá mô hình: sau khi đã huấn luyện mô hình xong, chúng ta cần đánh giá mô hình, tùy vào thông số được cài đặt thì độ chính xác cũng sẽ cho ra khác nhau.

Thứ năm, Improve – cải thiện: sau khi đã đánh giá mô hình, các mô hình đạt độ chính xác không tốt thì cần được train (rèn luyện) lại, chúng ta sẽ lặp lại từ bước 3, cho đến khi đạt độ chính xác như kỳ vọng. Tổng thời gian của 3 bước cuối rơi vào khoảng 30% tổng thời gian thực hiện.



Hình 2. Hình về quy trình làm việc với machine learning

1.2.2. Phân loại các mô hình máy học phổ biến

Có rất nhiều cách phân loại học máy - machine learning, thông thường thì machine learning sẽ được phân làm hai loại chính sau:

- *Supervised learning*: học có giám sát, là việc cho máy tính học trên dữ liệu đã được gán nhãn (label), hay nói cách khác, với mỗi đầu vào được đánh dấu là X_i , chúng ta sẽ có nhãn Y_i tương ứng. Ví dụ điển hình là các thuật toán như Support Vector Machines (SVM), K-Nearest Neighbors (KNN), và Logistic Regression. Các mô hình này yêu cầu có dữ liệu đã được gán nhãn (labelled data).

- *Unsupervised learning*: học không giám sát, là cho máy tính học trên dữ liệu mà không được gán nhãn, các thuật toán machine learning sẽ tìm ra sự tương quan dữ liệu, mô hình hóa dữ liệu hay chính là làm cho máy tính có kiến thức, hiểu về dữ liệu, từ đó chúng có thể phân loại các dữ liệu về sau thành các nhóm, lớp (clustering) giống nhau mà chúng đã được học hoặc giảm số chiều dữ liệu (dimension reduction). Các thuật toán như K-means clustering hoặc Hierarchical clustering không yêu cầu dữ liệu đã gán nhãn.

Ngoài ra, machine learning còn có thể phân làm các loại sau:

- *Semi-supervised learning*: học bán giám sát

- *Deep learning*: học sâu, đây cũng là một loại phổ biến được ứng dụng đối với cách dữ liệu yêu cầu sự phức tạp và độ chính xác cao. Các mô hình như Convolutional Neural Networks (CNN) và Recurrent Neural Networks (RNN) được sử dụng trong các bài toán phức tạp như nhận diện hình ảnh và xử lý ngôn ngữ tự nhiên.

- *Reinforce learning*: học củng cố/tăng cường

Một trong các mô hình (*model*) học máy phổ biến trong nhận diện hình ảnh và phân loại mà đề tài này sẽ sử dụng là Random Forest dựa trên sự phân loại (Classifier), đây là một mô hình thuộc nhóm Supervised learning và các chữ cái hay kí tự sẽ được gán các nhãn (label). Thuật toán này hoạt động bằng cách tạo ra nhiều cây quyết định (decision trees) độc lập và sử dụng kết quả từ các cây này để đưa ra một dự đoán cuối cùng, từ đó tăng độ chính xác và giảm thiểu hiện tượng overfitting.

1.2.3. Giới thiệu về Scikit-learn

Scikit-learn là một thư viện mã nguồn mở mạnh mẽ dành cho học máy (machine learning) trong Python, được xây dựng trên nền tảng của các thư viện khoa học phổ biến như NumPy, SciPy và matplotlib. Được thiết kế với mục tiêu dễ sử dụng và hiệu quả, Scikit-learn còn cung cấp một loạt các công cụ để giải quyết các bài toán học máy phổ biến, bao gồm: Phân loại (Classification). Hồi quy (Regression). Phân cụm (Clustering). Giảm chiều dữ liệu (Dimensionality Reduction). Kiểm định thống kê (Model Selection). Tiền xử lý dữ liệu (Preprocessing).

Các đặc điểm nổi bật của Scikit-learn:

- *Giao diện thân thiện*: Scikit-learn cung cấp các API trực quan, dễ học và sử dụng, ngay cả với người mới bắt đầu học máy. Mô hình học máy có thể được xây dựng, huấn luyện và đánh giá chỉ qua vài dòng mã.

- *Đa dạng thuật toán:* Thư viện này tích hợp hầu hết các thuật toán học máy quan trọng như Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Decision Trees và nhiều thuật toán khác.

- *Khả năng mở rộng:* Nhờ tích hợp với NumPy, Scikit-learn có thể xử lý hiệu quả dữ liệu lớn mà không cần đến tài nguyên phần cứng chuyên dụng như GPU.

- *Hỗ trợ tiền xử lý dữ liệu:* Các công cụ tích hợp sẵn giúp chuẩn hóa, chuẩn bị và chuyển đổi dữ liệu (như MinMaxScaler, StandardScaler) để tối ưu hóa hiệu quả của các thuật toán học máy.

- *Tích hợp tốt với các công cụ khác:* Scikit-learn hoạt động liền mạch với các thư viện Python khác, đặc biệt là Pandas cho quản lý dữ liệu và Matplotlib để trực quan hóa.

Trong khuôn khổ đề tài nhận diện cử chỉ tay ASL, Scikit-learn sẽ được dùng để lấy mô hình thuật toán Random Forest Classifier. Đây là thuật toán chính được sử dụng trong bài toán phân loại cử chỉ. Scikit-learn cung cấp công cụ dễ dàng để xây dựng, huấn luyện và tối ưu hóa mô hình Random Forest.

Scikit-learn không chỉ là một thư viện học máy mạnh mẽ mà còn là một công cụ hỗ trợ toàn diện trong toàn bộ chu trình xây dựng và triển khai mô hình. Sự tích hợp dễ dàng, các thuật toán phong phú và công cụ tiền xử lý hiệu quả khiến Scikit-learn trở thành một phần không thể thiếu trong nghiên cứu và phát triển các bài toán học máy, đặc biệt là trong bài toán nhận diện cử chỉ ASL.

1.3. Mediapipe và ứng dụng trong nhận diện bàn tay

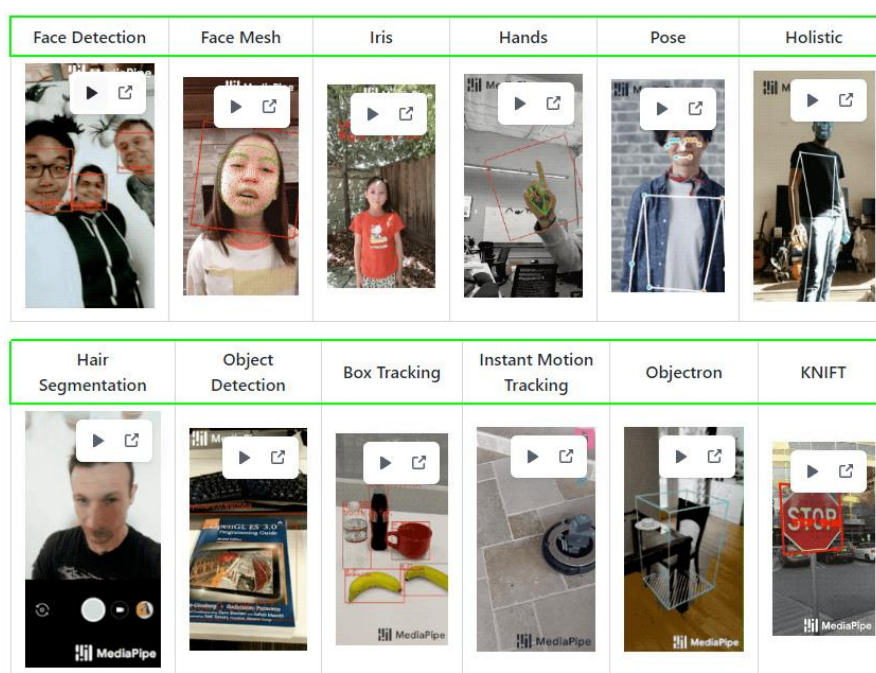
1.3.1. Giới thiệu về mediapipe

Mediapipe là một thư viện mã nguồn mở do Google phát triển, cung cấp các giải pháp mạnh mẽ cho việc nhận diện và xử lý các tín hiệu hình ảnh và video. Mediapipe hỗ trợ nhận diện nhiều đối tượng như khuôn mặt, cơ thể, bàn tay, và các hoạt động. Thư viện này cung cấp các mô hình đã được huấn luyện sẵn, cho phép người dùng dễ dàng triển khai các hệ thống nhận diện mà không cần phải xây dựng mô hình từ đầu. Một số ưu điểm có thể kể tới của mediapipe bao gồm:

- *Cung cấp một giải pháp inference nhanh chóng*: Google khẳng định rằng bộ công cụ này có thể chạy ổn định trên hầu hết các cấu hình phần cứng thông dụng.

- *Đễ dàng cài đặt và triển khai*: Việc cài đặt cực kỳ dễ dàng và tiện lợi, có thể triển khai trên nhiều nền tảng khác nhau như Mobile (Android/iOS), Desktop/Cloud, Web và IoT devices.

- *Mã nguồn mở và miễn phí*: Toàn bộ source code được công khai trên MediaPipe, người dùng có thể sử dụng và tùy chỉnh trực tiếp để phù hợp với bài toán của mình.



Hình 3. Hình về các loại nhận dạng phổ biến của mediapipe

1.3.2. Triển khai nhận diện tay bằng mediapipe

Hands Detection, hay còn gọi là nhận diện bàn tay, là bài toán mà chúng ta sẽ thử nghiệm trong đề tài này. Đầu ra của giải pháp này là một mô hình skeleton (khung xương) của bàn tay, gồm vị trí của các landmarks trên bàn tay và được nối với nhau thành một khung bàn tay hoàn chỉnh.

Mediapipe cung cấp một mô - đun nhận diện bàn tay có thể phát hiện các điểm then chốt trên bàn tay (21 đốt) và các đường nối giữa chúng để tạo thành cấu trúc xương tay (3D-Skeleton), từ đó xác định các cử chỉ và hành động. Quá trình này bao gồm:

- *Phát hiện bàn tay*: Xác định vị trí và kích thước của bàn tay trong ảnh/video.

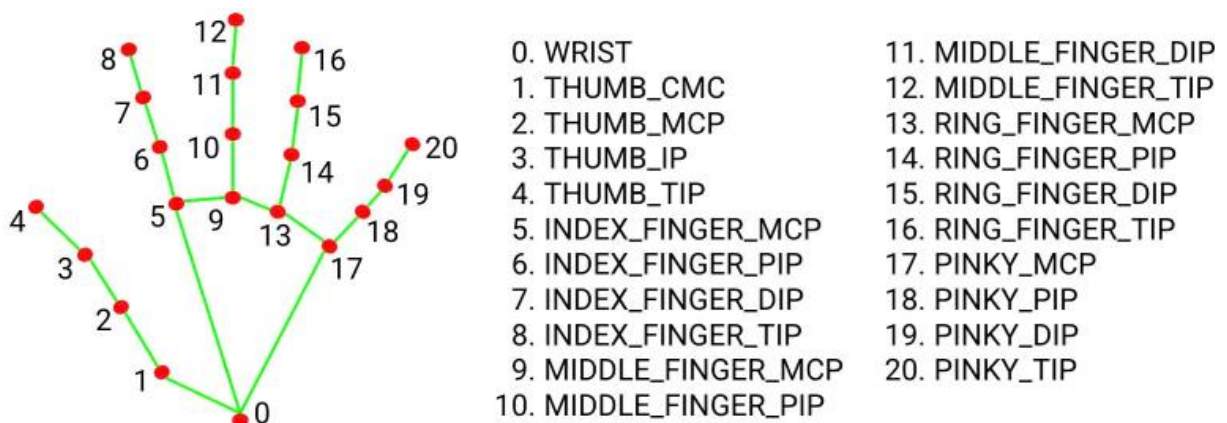
- *Đánh dấu các điểm then chốt và nối các điểm*: Các điểm đặc trưng của bàn tay như đầu ngón tay, khớp tay, được đánh dấu để theo dõi chuyển động.

- *Xử lý và phân tích các cử chỉ*: Các cử chỉ tay có thể được phân loại, ví dụ như ngón tay giơ lên, nắm tay, hoặc các ký hiệu khác trong ngôn ngữ ký hiệu.

Ứng dụng của Mediapipe trong nhận diện bàn tay rất hữu ích trong các lĩnh vực như:

- *Hệ thống nhận diện ngôn ngữ ký hiệu*: Có thể giúp dịch các ký hiệu ASL thành văn bản hoặc giọng nói.

- *Điều khiển tương tác người-máy*: Cử chỉ tay có thể được sử dụng để điều khiển các thiết bị hoặc giao diện người dùng.



Hình 4. Hình mô phỏng cấu trúc xương tay

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG VÀ CÁC THƯ VIỆN BỔ SUNG

2.1. Thiết kế hệ thống

2.1.1. Kiến trúc tổng quan

Hệ thống được thiết kế theo quy trình tuần tự sau đây: Thu thập hình ảnh bàn tay từ camera, chuyển đổi hình ảnh sang dữ liệu số thông qua các kỹ thuật xử lý ảnh, sau đó sử dụng dữ liệu đã xử lý này để huấn luyện mô hình máy học. Cuối cùng, hệ thống sẽ sử dụng mô hình máy học đã huấn luyện để nhận dạng dữ liệu từ bàn tay. Kiến trúc này đảm bảo tính hệ thống và dễ dàng nâng cấp trong tương lai.

2.1.2. Thiết kế mô hình học máy Random Forest

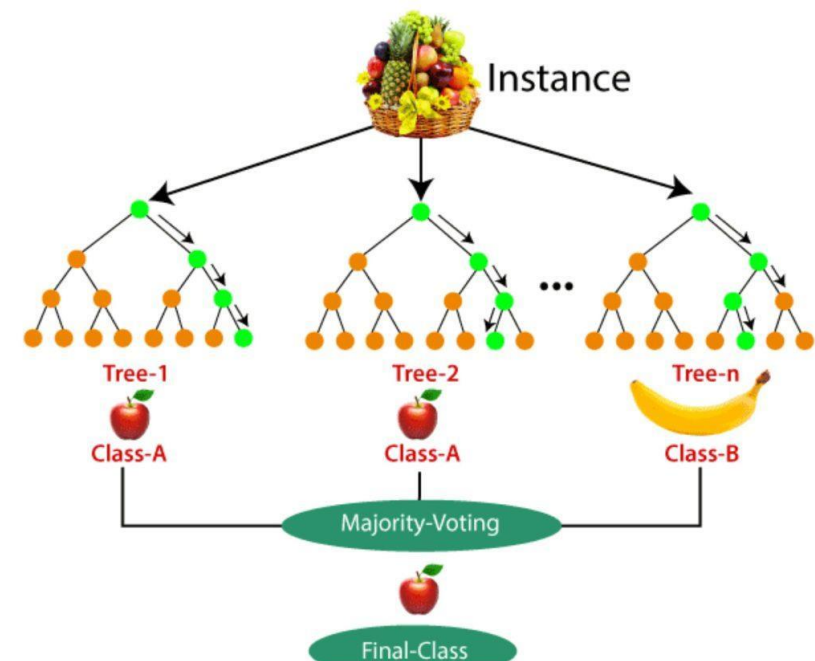
Mô hình học máy được sử dụng trong đề án này là *Random Forest*. Lựa chọn này dựa trên khả năng xử lý dữ liệu hình ảnh hiệu quả, tính ổn định và đơn giản hơn so với các mô hình khác. Random Forest là một phương pháp học tập tổng hợp (ensemble learning) kết hợp nhiều cây quyết định, giúp tăng cường độ chính xác và độ ổn định của mô hình. Trong hệ thống này, chúng em sử dụng Random Forest với 200 cây quyết định ($n_estimators$) và độ sâu tối đa ($tree_depth$) là 35. Các tham số này được lựa chọn dựa trên kết quả thực nghiệm và tối ưu hóa trên tập dữ liệu huấn luyện. Việc sử dụng nhiều cây quyết định giúp giảm thiểu ảnh hưởng của các cây quyết định yếu và tăng cường khả năng tổng quát hóa của mô hình. Độ sâu tối đa được giới hạn để tránh overfitting và đảm bảo mô hình có thể hoạt động tốt trên dữ liệu mới.

Random Forest là gì? *Rừng Ngẫu nhiên (Random Forest)* là một phương pháp mạnh mẽ trong học máy, hoạt động bằng cách kết hợp nhiều "*cây quyết định*" để đưa ra dự đoán. Hãy tưởng tượng mỗi cây quyết định như một chuyên gia đưa ra quyết định dựa trên một số thông tin nhất định. Cụ thể hơn trong quá trình huấn luyện, Rừng Ngẫu nhiên tạo ra nhiều cây quyết định, mỗi cây được "*huấn luyện*" trên một phần ngẫu nhiên của dữ liệu và chỉ xem xét một số đặc trưng ngẫu nhiên. Việc này giống như việc giao cho mỗi chuyên gia một phần công việc và yêu cầu họ tập trung vào một số khía cạnh nhất định. Nhờ tính ngẫu nhiên này, mỗi cây quyết định sẽ có một góc nhìn khác nhau về dữ liệu, tránh tình trạng tất cả đều mắc cùng một sai lầm (giảm thiểu quá khớp - overfitting).

Khi cần đưa ra dự đoán, Random Forest sẽ hỏi ý kiến của tất cả các cây quyết định. Nếu là bài toán phân loại (ví dụ: phân loại ảnh là táo hay chuối), kết quả cuối cùng sẽ

được quyết định bằng cách "*bỏ phiếu*" - lấy kết quả được nhiều cây dự đoán nhất. Còn nếu là bài toán hồi quy (ví dụ: dự đoán giá nhà), kết quả sẽ là giá trị trung bình của dự đoán từ tất cả các cây.

Tóm lại, Random Forest hoạt động theo nguyên tắc "*Chọn Theo Số Đông*": kết hợp sức mạnh của nhiều cây quyết định để đưa ra dự đoán chính xác và ổn định hơn, ngay cả khi dữ liệu phức tạp.



Hình 5. Hình ví dụ về mô hình Random Forest

2.1.3. Thiết kế giao diện

Giao diện của đề tài được thiết kế bằng cách sử dụng Tkinter và CustomTkinter.

Tkinter là thư viện chuẩn của Python cho việc xây dựng giao diện người dùng (GUI), cung cấp một tập hợp các widget cơ bản và dễ sử dụng. Là một phần của thư viện chuẩn, Tkinter có sẵn trên hầu hết các bản cài đặt Python, giúp việc bắt đầu phát triển GUI trở nên đơn giản và thuận tiện. Tuy nhiên, giao diện mặc định của Tkinter thường bị cho là đơn giản và có phần lỗi thời so với các framework GUI hiện đại khác.

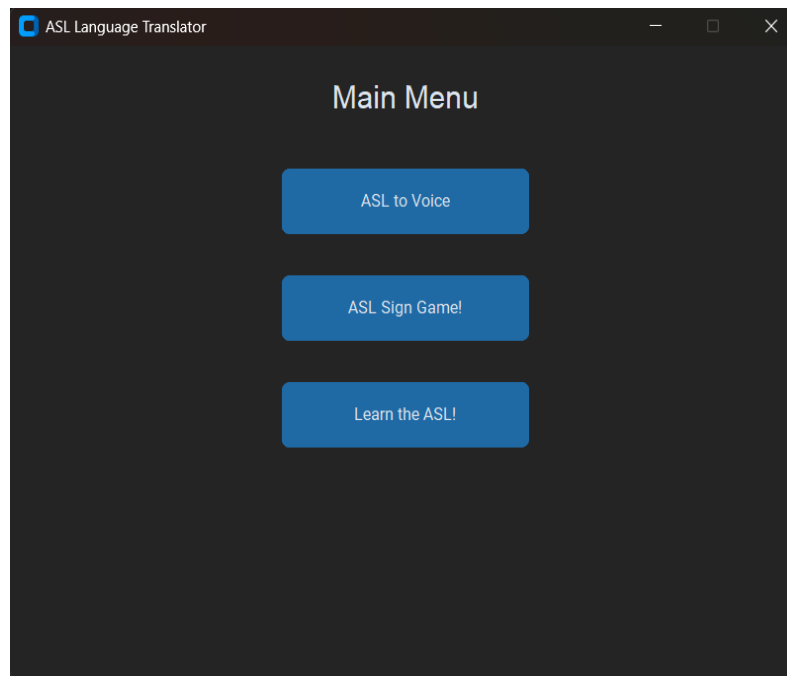
CustomTkinter là một thư viện được xây dựng dựa trên Tkinter, nhằm mục đích cải thiện giao diện và trải nghiệm người dùng. Nó cung cấp một bộ các widget được tùy chỉnh và phong cách hiện đại, cho phép người dùng tạo ra các ứng dụng GUI đẹp mắt và chuyên nghiệp hơn mà không cần phải từ bỏ sự đơn giản và dễ sử dụng của Tkinter.

CustomTkinter đơn giản hóa việc tùy chỉnh giao diện bằng cách cung cấp các tùy chọn cấu hình trực quan cho màu sắc, font chữ, hình dạng nút bấm và nhiều yếu tố giao diện khác. Hơn nữa, CustomTkinter hỗ trợ cả giao diện sáng và tối, cũng như tích hợp liền mạch với các theme màu sắc khác nhau.

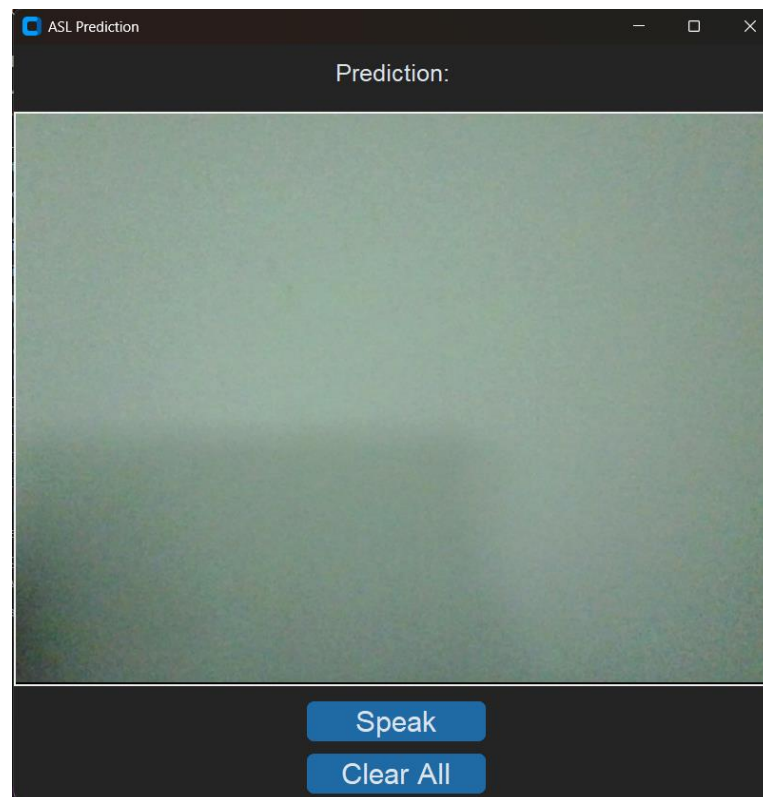
Sự kết hợp giữa Tkinter và CustomTkinter mang lại một giải pháp lý tưởng cho việc phát triển GUI trong Python. Tkinter cung cấp nền tảng vững chắc và dễ tiếp cận, trong khi CustomTkinter nâng cao tính thẩm mỹ và trải nghiệm người dùng. Đặc biệt, CustomTkinter giảm thiểu đáng kể nỗ lực cần thiết để tùy chỉnh giao diện so với việc sử dụng Tkinter, cho phép nhà phát triển tập trung vào chức năng của ứng dụng. Nhờ đó, CustomTkinter là một lựa chọn tuyệt vời cho cả người mới bắt đầu và những người đã có kinh nghiệm với Tkinter, mong muốn tạo ra các ứng dụng GUI hiện đại và hấp dẫn. Ngoài ra, CustomTkinter còn tương thích với nhiều hệ điều hành, đảm bảo tính khả chuyển của ứng dụng. Có thể nói, việc lựa chọn thư viện Tkinter và CustomTkinter cho phát triển giao diện người dùng (GUI) của dự án mang lại nhiều lợi ích đáng kể.

Cụ thể, CustomTkinter cung cấp các widget được thiết kế lại với giao diện hiện đại, tuân thủ các nguyên tắc thiết kế UI/UX đương đại, giúp nâng cao trải nghiệm người dùng. Việc sử dụng CustomTkinter giúp tiết kiệm thời gian và công sức phát triển giao diện nhờ các thành phần được tùy chỉnh sẵn, cho phép lập trình viên tập trung vào logic nghiệp vụ của ứng dụng.

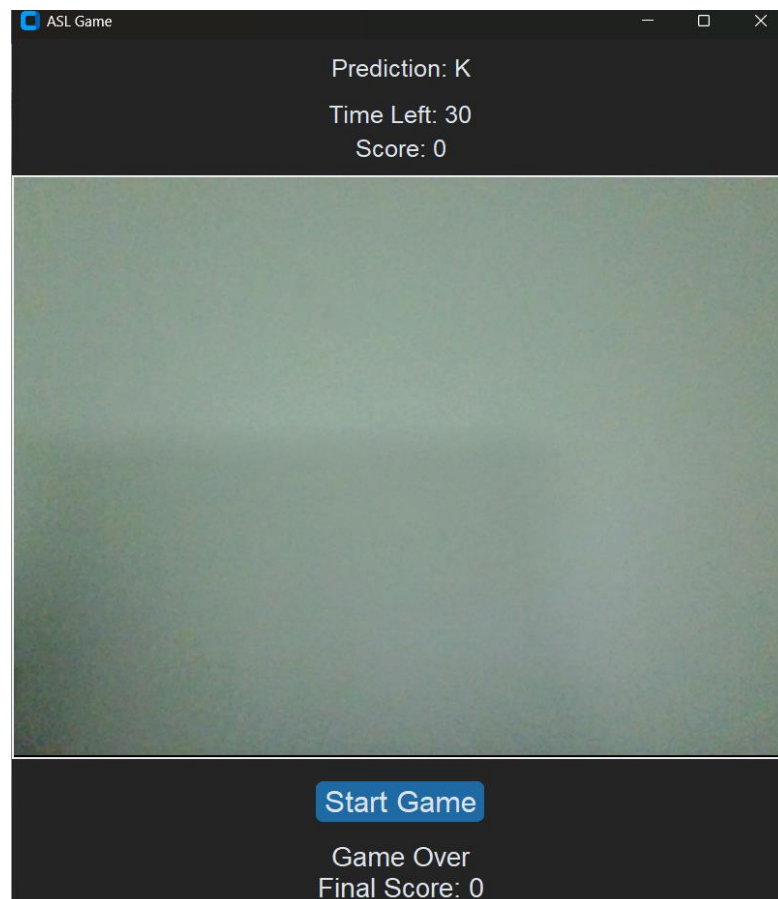
Sự kết hợp này mang lại một giao diện người dùng vừa hiện đại, dễ sử dụng, vừa đảm bảo tính ổn định và hiệu năng. Việc sử dụng các thư viện Python phổ biến cũng giúp dễ dàng tích hợp với các module khác và đơn giản hóa quá trình bảo trì, cập nhật.



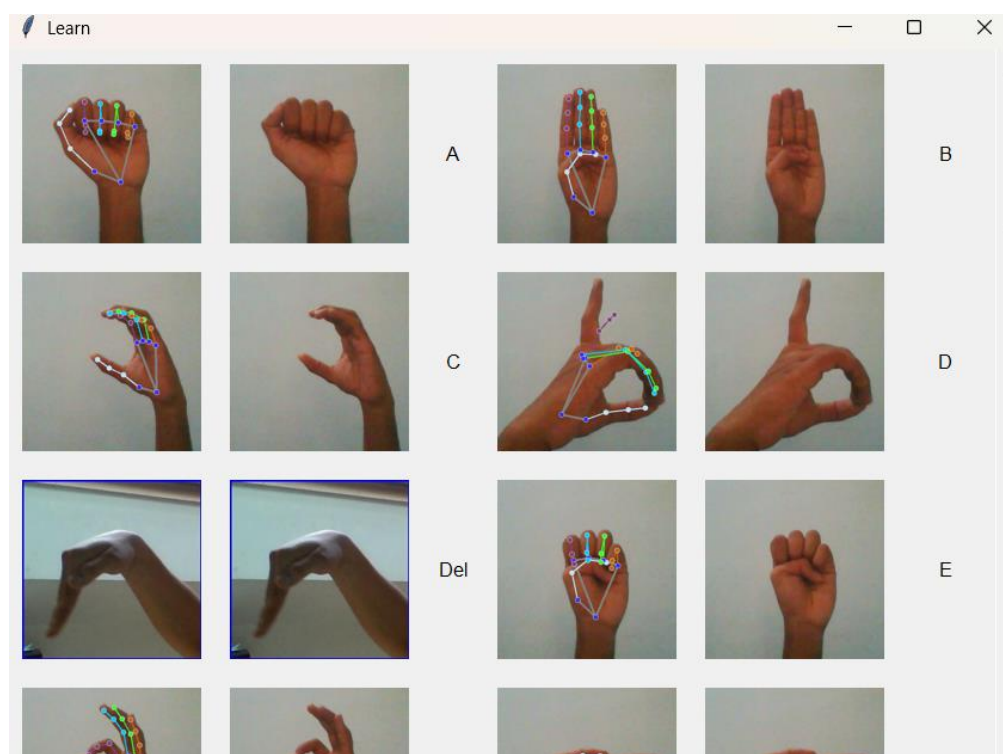
Hình 6. Hình giao diện menu



Hình 7. Hình giao diện ASL to Voice



Hình 8. Hình giao diện ASL Sign Game



Hình 9. Hình giao diện Learn The ASL

2.2. Công cụ và thư viện khác

2.2.1. OpenCV

OpenCV là gì? *OpenCV (Open Source Computer Vision Library)* là một thư viện mã nguồn mở mạnh mẽ, cung cấp một tập hợp toàn diện các công cụ và thuật toán cho thị giác máy tính, học máy và xử lý ảnh. Vai trò của OpenCV đặc biệt quan trọng trong các ứng dụng thời gian thực, vốn là yêu cầu thiết yếu của nhiều hệ thống hiện đại. Trong dự án này, OpenCV được sử dụng để xử lý hình ảnh bàn tay, cho phép nhận dạng và phân tích cử chỉ.

Khả năng xử lý hình ảnh và video của OpenCV cho phép hệ thống nhận dạng các đối tượng, khuôn mặt, và thậm chí cả chữ viết tay. Việc tích hợp OpenCV với NumPy, một thư viện Python mạnh mẽ cho phép tính toán số, cho phép xử lý hiệu quả cấu trúc mảng của OpenCV. Sự kết hợp này tạo điều kiện cho việc phân tích hình ảnh bằng cách sử dụng không gian vector và thực hiện các phép toán trên các đặc trưng của ảnh, từ đó xác định các mẫu và đặc điểm của bàn tay. Điều này đóng vai trò then chốt trong việc xây dựng một hệ thống nhận dạng cử chỉ bàn tay chính xác và hiệu quả.

Trong đề tài nhận dạng cử chỉ tay ASL, OpenCV đóng vai trò quan trọng trong các giai đoạn sau:

- *Xử lý ảnh đầu vào*: OpenCV được sử dụng để đọc và tiền xử lý các khung hình từ camera. Điều này bao gồm chuyển đổi ảnh sang không gian màu cần thiết, cắt ảnh để tập trung vào vùng bàn tay và giảm nhiễu để cải thiện hiệu quả của các bước xử lý tiếp theo.

- *Phát hiện và phân tích đặc trưng*: OpenCV hỗ trợ việc phát hiện các điểm đặc trưng trên bàn tay bằng cách sử dụng các thuật toán xử lý hình ảnh như phân vùng theo ngưỡng (thresholding), phân tích đường viền (contour analysis), và phát hiện đặc trưng hình học.

- *Tích hợp với Mediapipe*: OpenCV được kết hợp cùng Mediapipe để trích xuất các đặc trưng bàn tay (landmarks). Các khung hình được xử lý qua OpenCV trước khi được đưa vào Mediapipe nhằm tăng độ chính xác của hệ thống.

- *Hỗ trợ thời gian thực*: Khả năng xử lý thời gian thực của OpenCV đảm bảo rằng hệ thống nhận dạng cử chỉ hoạt động hiệu quả với độ trễ thấp, đáp ứng được yêu cầu của bài toán thực tế.

2.2.2. Pandas và Pickle

Pandas là gì? *Pandas* là một thư viện Python cốt lõi cho phân tích và thao tác dữ liệu, nổi tiếng với hiệu suất cao và tính linh hoạt. Nó cung cấp các cấu trúc dữ liệu giàu tính năng như *DataFrame* và *Series*, tối ưu hóa cho việc xử lý dữ liệu dạng bảng và chuỗi thời gian. *DataFrame*, đặc biệt, cho phép tổ chức dữ liệu theo dạng bảng với nhãn cho cả hàng và cột, hỗ trợ các phép toán phức tạp một cách hiệu quả. Khả năng đọc và ghi dữ liệu từ nhiều định dạng khác nhau (như CSV, Excel, SQL databases) khiến *Pandas* trở thành công cụ không thể thiếu trong quy trình làm việc với dữ liệu. Hơn nữa, khả năng tích hợp liền mạch với các thư viện khoa học dữ liệu khác trong hệ sinh thái Python (như *Scikit-learn*, *NumPy*) càng củng cố vị thế quan trọng của *Pandas*.

Pickle là một mô-đun trong Python được thiết kế để tuần tự hóa và giải tuần tự hóa các đối tượng Python. Quá trình này, thường được gọi là "*pickling*" và "*unpickling*", biến đổi một đối tượng Python thành một chuỗi byte có thể lưu trữ được (ví dụ: trong một tệp) và sau đó khôi phục lại đối tượng ban đầu từ chuỗi byte đó. Điều này cực kỳ hữu ích cho việc lưu trữ các trạng thái của chương trình, bao gồm cả các mô hình học máy đã được huấn luyện, tránh việc phải huấn luyện lại tốn kém về thời gian và tài nguyên. Tuy nhiên, cần lưu ý rằng *Pickle* chủ yếu dành cho việc tuần tự hóa các đối tượng Python và không được khuyến nghị sử dụng cho việc trao đổi dữ liệu giữa các hệ thống khác nhau hoặc các phiên bản Python khác nhau do các vấn đề về tương thích và bảo mật tiềm ẩn. Đối với các tình huống này, các định dạng tuần tự hóa khác như JSON hoặc HDF5 thường được ưu tiên hơn.

Trong ngữ cảnh học máy, *Pandas* và *Pickle* thường được sử dụng kết hợp. *Pandas* được sử dụng để chuẩn bị và xử lý dữ liệu huấn luyện, thường được lưu trữ dưới dạng *DataFrame*. Sau khi mô hình được huấn luyện, *Pickle* được sử dụng để lưu trữ mô hình đó vào đĩa. Việc này cho phép tải lại mô hình đã huấn luyện một cách nhanh chóng mà không cần phải lặp lại quá trình huấn luyện, giúp tiết kiệm thời gian và tài nguyên tính toán đáng kể. Ví dụ, trong một quy trình triển khai mô hình, việc sử dụng *Pickle* để lưu

và tải mô hình là một bước quan trọng để đảm bảo hiệu quả và khả năng mở rộng. Tóm lại, sự kết hợp giữa Pandas và Pickle tạo nên một quy trình làm việc hiệu quả và mạnh mẽ cho việc phát triển và triển khai các ứng dụng học máy.

2.2.3. Google.generativeai

Google Generative AI là một công cụ tiên tiến giúp tạo ra nội dung thông minh dựa trên các mô hình ngôn ngữ lớn. Trong dự án này, công cụ này sử dụng API của một mô hình AI là Gemini. Việc ứng dụng AI vào là để xử lý chủ động các văn bản hoặc ký tự sao cho hợp lý và mạch lạc nhất và xử lý các trường hợp đặc biệt như:

- *Sai chính tả và ngữ pháp*: AI có thể sửa lại các câu sai chính tả và ngữ pháp và chuyển thành một câu hoàn chỉnh giúp cho đầu ra luôn luôn đúng.

- *Từ hoặc cụm từ viết tắt*: Chuyển các từ hoặc cụm từ viết tắt sang một từ hoặc cụm từ hoàn chỉnh sẽ làm cho câu văn trở nên chính xác hơn.

- *Tích hợp ngữ cảnh*: Tự xử lý ngữ cảnh câu sao cho phù hợp và có thể giảm thiểu tình trạng sai sót hoặc thiếu chi tiết quan trọng.

Trong đề tài này, sau khi người dùng tạo ra các văn bản từ các ngôn ngữ ký hiệu tay, AI sẽ chuyển các từ ngữ đó thành một câu hoàn chỉnh. Điều này giúp người dùng có thể viết ngắn gọn và đưa vào các từ ngữ viết tắt mà không bị mất đi ý nghĩa của toàn bộ điều muốn nói từ câu văn đầu vào.

2.2.4. gTTS

gTTS (Google Text-to-Speech) là một công cụ Python mã nguồn mở, được sử dụng để chuyển văn bản thành giọng nói. Trong dự án này, gTTS đóng vai trò quan trọng trong việc cung cấp phản hồi âm thanh, tạo điều kiện giao tiếp tốt hơn cho người dùng:

- *Chuyển đổi văn bản thành âm thanh*: gTTS hỗ trợ hệ thống phát âm các cử chỉ được nhận diện thành lời nói, giúp người dùng có thể giao tiếp qua âm thanh một cách tự nhiên.

- *Hỗ trợ đa ngôn ngữ*: Công cụ này hỗ trợ nhiều ngôn ngữ, bao gồm cả tiếng Anh và tiếng Việt, đảm bảo tính linh hoạt trong các ứng dụng thực tế.

- *Tích hợp thời gian thực*: Việc tích hợp gTTS trong hệ thống đảm bảo rằng âm thanh được phát ngay lập tức sau khi cử chỉ được nhận diện, tạo trải nghiệm liền mạch và hiệu quả.

Vì là bảng ngôn ngữ ASL nên gTTS sẽ được cài với ngôn ngữ giọng tiếng Anh (en) và chúng ta sẽ sử dụng pygame để có thể xuất ra âm thanh đến từ file âm thanh mà gTTS đã cung cấp.

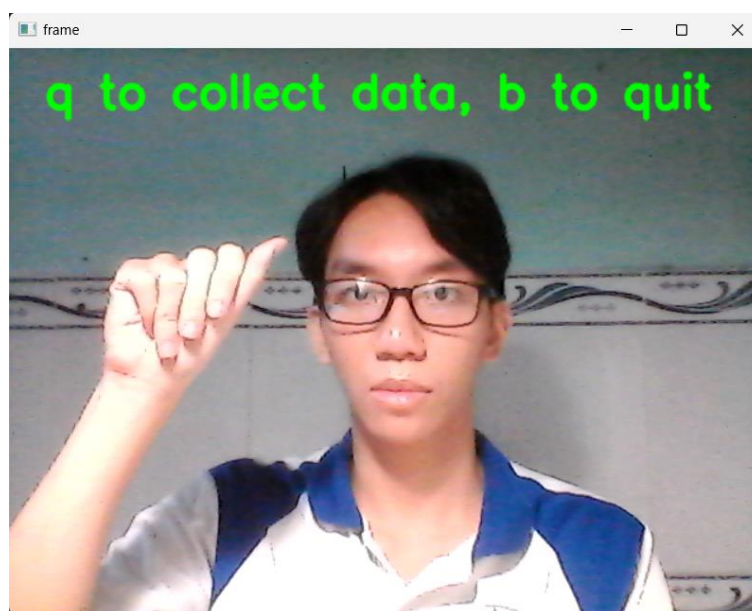
CHƯƠNG 3: TRIỂN KHAI, CẢI TIẾN VÀ THỰC NGHIỆM

3.1. Dữ liệu và xử lý dữ liệu

3.1.1. Thu thập dữ liệu ASL Dataset

Đầu tiên là quá trình thu thập dữ liệu, trong quá trình này sẽ sử dụng một thư viện chính đó chính là OpenCV. Quá trình lấy dữ liệu có thể được mô phỏng dựa trên các bước như sau:

- *Bước 1:* Dùng OpenCV để mở camera chính, sau đó chuẩn bị một đường dẫn để chỉ đến thư mục thu thập dữ liệu



```
while True:
    ret, frame = cap.read()
    cv2.putText(frame, 'q to collect data, b to quit', (30, 50), cv2.FONT_HERSHEY_SIMPLEX, 1.3, (0,255,0), 3, cv2.LINE_AA)
    cv2.imshow('frame', frame)
    key = cv2.waitKey(25) & 0xFF
    if key == ord('q'): # Start collecting data
        break
    elif key == ord('b'): # Stop everything
        print("Stopping data collection...")
        cap.release()
        cv2.destroyAllWindows()
        exit()

counter = 0
while counter < dataset_size:
    ret, frame = cap.read()
    cv2.imshow('frame', frame)
    cv2.waitKey(25)
    cv2.imwrite(os.path.join(DATA_DIR, str(j), '{}.jpg'.format(counter)), frame)

    counter += 1
```

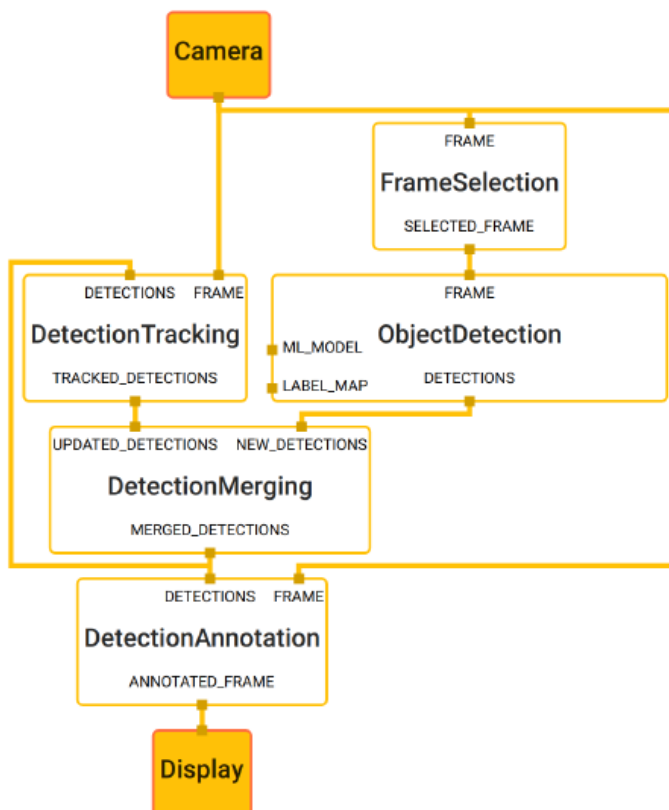
Hình 10. Hình ảnh mô phỏng về quá trình thu thập dữ liệu qua OpenCV

- *Bước 2*, bắt đầu quá trình thu thập dữ liệu: Khi nhấn nút “q”, OpenCV sẽ thực hiện lấy các frame từ camera và từ đó đưa vào trong nơi chứa hình ảnh nhân đang được lấy dữ liệu. Ví dụ: Mô phỏng cho việc lấy dữ liệu nhãn 0, lưu trong thư mục data.



Hình 11. Hình ảnh mô phỏng thư mục lưu trữ dữ liệu

Dữ liệu hình ảnh được gọi là *dữ liệu đã đánh nhãn (Annotated/Labeled Image)*. Ngoài ra, các hình ảnh trong dữ liệu có thể đem cắt chỉ để lấy hình ảnh bàn tay, giúp cho việc nhận dạng càng trở nên chính xác hơn. Trong đề tài này, dữ liệu sẽ được lấy từ bộ dữ liệu dataset có sẵn trên mạng (kaggle) bao gồm tổng cộng 26 *nhãn ký tự chữ cái* và 3 *nhãn đặc biệt* gồm xóa (Del), cách (Space) và không có gì (Nothing).



Hình 12. Sơ đồ cho quá trình thu thập hình ảnh và đánh nhãn

3.1.2. Xử lý dữ liệu

Thứ hai là quá trình xử lý dữ liệu thu thập từ bộ dữ liệu ký hiệu ngôn ngữ Mỹ (American Sign Language - ASL)., quá trình này sẽ kết hợp thêm thư viện Mediapipe hỗ trợ phát hiện và theo dõi các điểm của cấu trúc xương bàn tay, được tích hợp để tạo ra dữ liệu đặc trưng cho mô hình nhận diện ký hiệu. Thông qua mp.solutions.hands, thư viện này xác định chính xác 21 điểm landmarks của bàn tay, bao gồm các khớp ngón tay và lòng bàn tay, ngay cả khi tay chuyển động hoặc thay đổi góc độ.

Để mô hình có thể nhận diện các ký hiệu, việc chuẩn hóa dữ liệu và chuyển đổi chúng sang định dạng đầu vào phù hợp là rất quan trọng. Các bước xử lý bao gồm:

- *Ghi nhận tọa độ*: Chúng ta sẽ dùng OpenCV để chuyển hình ảnh từ dạng RGB sang BGR để Mediapipe có thể trích xuất các đặc trưng từ các điểm (landmarks) của bàn tay. Quá trình chuyển đổi hình ảnh vì Mediapipe nhận diện các điểm đối với hình ảnh BGR. Mỗi điểm bao gồm tọa độ (x, y) của các khớp (đốt) của bàn tay.

- *Làm sạch dữ liệu*: Loại bỏ các khung hình bị lỗi hoặc không đủ dữ liệu (như bàn tay không xuất hiện hay có 2 bàn tay trong cùng một khung hình). Thêm vào đó, Mediapipe còn tích hợp các bộ lọc để loại bỏ các khung hình không chứa bàn tay hoặc có độ tin cậy thấp (*min_detection_confidence*), giảm thiểu nhiễu và tối ưu hóa hiệu quả xử lý. Quy trình này giúp đảm bảo tập dữ liệu đầu ra có chất lượng cao, đáp ứng tốt yêu cầu huấn luyện mô hình nhận diện ký hiệu ngôn ngữ.

- *Chuẩn hóa dữ liệu*: Chuyển đổi tọa độ về một khoảng giá trị cố định để đảm bảo tính đồng nhất giữa các mẫu. Sau khi phát hiện các landmarks, Mediapipe sử dụng *mp_drawing* để kết nối các điểm này, tạo thành cấu trúc bàn tay trực quan trên khung hình. Dữ liệu tọa độ x, y của từng điểm được chuẩn hóa về phạm vi [0, 1] dựa trên kích thước khung hình, đảm bảo sự nhất quán khi áp dụng trên các thiết bị khác nhau. Mỗi khung hình sẽ được chuyển thành một vector gồm 42 giá trị (21 điểm x 2 tọa độ), giúp mô hình học máy nhận biết đầy đủ vị trí và cấu trúc bàn tay.

```

for j in os.listdir(os.path.join(data_dir,i)): #RGB -> BGR
    data_aux = [] #42
    img = cv2.imread(os.path.join(data_dir,i,j))
    img_rgb = cv2.cvtColor(img,cv2.COLOR_BGR2RGB)

    results = hands.process(img_rgb)
    if results.multi_hand_landmarks:
        for hand_landmarks in results.multi_hand_landmarks:
            for z in range(len(hand_landmarks.landmark)):
                x = hand_landmarks.landmark[z].x
                y = hand_landmarks.landmark[z].y
                data_aux.append(x)
                data_aux.append(y)

            data.append(data_aux)
            labels.append(i)

processed_files += 1
percent_complete = (processed_files / total_files) * 100
print(f"Processing: {percent_complete:.2f}% completed", end='\r')

```

Hình 13. Hình ảnh về quá trình xác định tọa độ cho cấu trúc tay

Cuối cùng, chúng ta sẽ có một bộ dữ liệu gồm các *nhãn (label)* ký tự và trong các nhãn đó sẽ chứa các giá trị $[0, 1]$ biểu thị cho tọa độ x và y của mỗi hình ảnh của nhãn. Dữ liệu này sẽ được dùng cho quá trình rèn luyện (training) các mô hình máy và các mô hình máy sẽ dự đoán các nhãn hay ký tự dựa trên tọa độ đó.

Đây là quá trình chiếm thời gian lâu nhất trong các quy trình rèn luyện mô hình máy học. Đối với một bộ dữ liệu training data lớn ví dụ như khoảng 5GB – 252670 tệp hình ảnh thì quy trình xử lý có thể lên tới 7 giờ đồng hồ!

3.1.3. Lưu trữ dữ liệu

Sau quá trình xử lý dữ liệu, chúng ta cũng có thể truyền thẳng dữ liệu vào trong quá trình rèn luyện mô hình. Nhưng điều này sẽ không có nhiều ưu điểm so với việc lưu dữ liệu đã xử lý lại rồi mới rèn mô hình như:

- *Tránh được rủi ro:* Quá trình xử lý dữ liệu chiếm phần lớn thời gian rèn luyện, vì vậy lưu lại sẽ tránh được các rủi ro trong quá trình training.

- *Có thể áp dụng để tạo ra nhiều mô hình khác nhau:* Khi mô hình rèn luyện sau, nếu như kết quả không như mong đợi thì có thể sử dụng lại bộ dữ liệu đã được xử lý chứ không cần phải xử lý lại từ đầu, giảm thiểu phần lớn thời gian.

- Có thể được chia sẻ và phát triển: Dữ liệu sau khi xử lý là các dữ liệu định dạng [0, 1] nên sẽ không chiếm quá nhiều dung lượng như dữ liệu hình ảnh ban đầu, nên có thể chia sẻ một cách nhanh chóng và tiện lợi.

Dữ liệu sau khi xử lý sẽ được lưu trữ dưới dạng file pandas (.csv) hoặc pickle (.p) tùy vào nhu cầu của người dùng, bao gồm các đặc trưng và nhãn tương ứng. Điều này giúp tăng tốc độ truy xuất dữ liệu trong quá trình huấn luyện và dự đoán.

```
df = pd.DataFrame(data)
df['label'] = labels
df.to_pickle('datatest.csv')

f = open('datatest.pickle', 'wb')
pickle.dump({'datatest':data, 'labels':labels},f)
f.close()
```

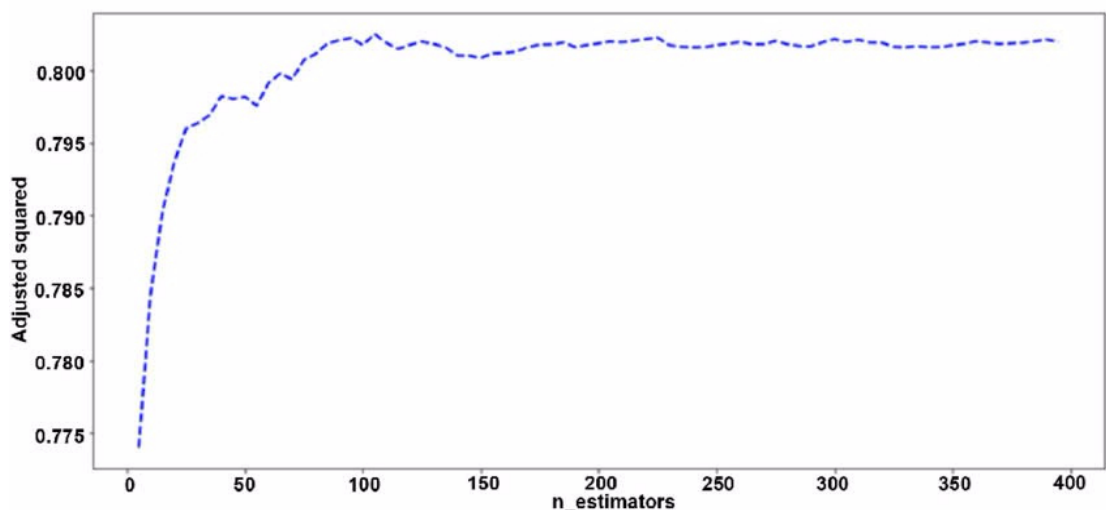
Hình 14. Hình ảnh về phương thức lưu trữ dữ liệu sau khi xử lý

3.2. Training mô hình

3.2.1. Các tham số đầu vào `n_estimators`, `max_depth`, `random_state`

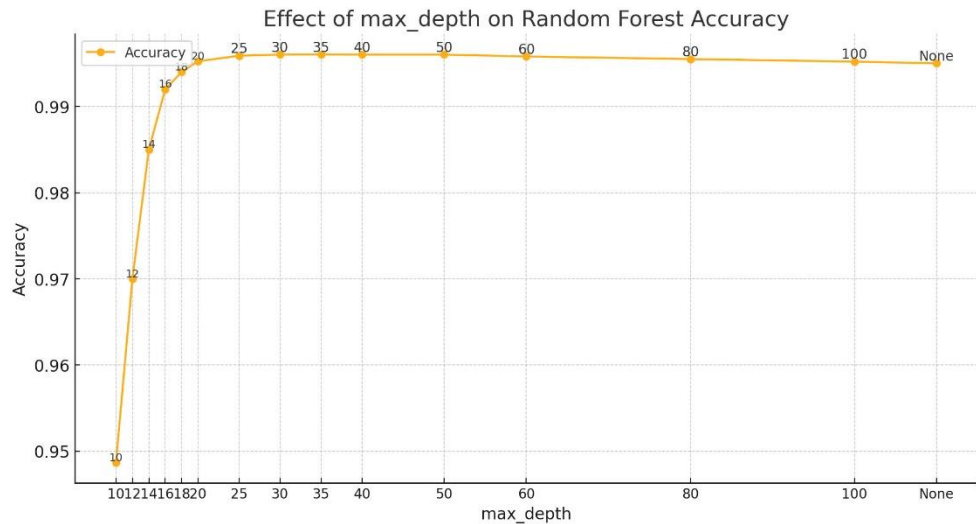
Mô hình Random Forest được lựa chọn để giải quyết bài toán nhận diện ngôn ngữ ký hiệu nhờ tính hiệu quả và khả năng xử lý tốt dữ liệu không đồng nhất. Trong quá trình thiết lập, các tham số quan trọng bao gồm:

- Tham số `n_estimators` được đặt là 200, đại diện cho số lượng cây trong rừng. Số lượng này được xác định thông qua thử nghiệm với GridSearch, cho thấy đây là con số cân bằng giữa độ chính xác và thời gian huấn luyện. Nếu số lượng cây quá ít, mô hình sẽ không đủ mạnh để học hết các đặc điểm của dữ liệu; ngược lại, nếu số lượng cây quá nhiều, thời gian tính toán sẽ tăng lên mà không cải thiện đáng kể hiệu quả.



Hình 15. Hình ảnh hưởng của `n_estimators` đến độ chính xác lý thuyết

- Tham số `max_depth`, được đặt là 35, quy định độ sâu tối đa của mỗi cây, qua đó kiểm soát mức độ phức tạp của mô hình. Giá trị này giúp mô hình đủ khả năng học các đặc điểm phức tạp từ dữ liệu mà không bị quá khớp (overfitting), đồng thời giữ thời gian huấn luyện ở mức hợp lý. Khi tăng `max_depth` vượt quá 35, kết quả thử nghiệm cho thấy độ chính xác không cải thiện nhiều, nhưng thời gian tính toán lại tăng đáng kể.



Hình 16. Hình ảnh hưởng của `max_depth` đến độ chính xác lý thuyết

- Tham số `random_state` được đặt là 42 để đảm bảo tính nhất quán và khả năng tái tạo của mô hình trong các lần chạy khác nhau. Việc chọn giá trị này không ảnh hưởng trực tiếp đến độ chính xác, nhưng rất quan trọng trong quá trình kiểm thử và so sánh.

Các tham số này đều được tối ưu hóa dựa trên dữ liệu từ ASL Dataset, đã qua xử lý với Mediapipe, để đạt được độ chính xác cao nhất có thể trên tập kiểm tra. Việc lựa chọn các giá trị cụ thể (200, 35, 42) không phải ngẫu nhiên mà xuất phát từ quá trình phân tích và thử nghiệm tỉ mỉ nhằm đảm bảo hiệu suất tốt nhất cho mô hình Random Forest. Việc điều chỉnh và tối ưu hóa các tham số trên là một phần không thể thiếu để đảm bảo mô hình đạt hiệu suất cao nhất trên bộ dữ liệu đầu vào.

3.2.2. Quá trình rèn luyện mô hình

Bắt đầu với việc lấy lại dữ liệu từ file dữ liệu đã được chuẩn hóa và lưu vào trong một mảng với các nhãn và data. Sau đó chúng ta sẽ thực hiện quá trình nạp dữ liệu vào trong model `RandomForestClassifier` với các thông số đầu vào.

Mô hình sau khi rèn luyện sẽ cung cấp ra độ chính xác lý thuyết trong quá trình rèn luyện đối với các dữ liệu đã được đánh nhãn trong điều kiện lý tưởng (không bị ngược sáng và nền trống).

```
data_dict = pd.read_pickle('datatest.csv')
data = data_dict.drop(columns=['label'])
labels = data_dict['label']

# Split data
X_train, X_test, y_train, y_test = train_test_split(np.array(data), labels, test_size=0.15, random_state=22, shuffle=True)
model = RandomForestClassifier(n_estimators=200, max_depth=35, random_state=42)
model.fit(X_train, y_train)

# Predict
pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, pred)
print(f'Accuracy: {accuracy * 100:.2f}%')

with open('modeltest.p', 'wb') as f:
    pickle.dump({'modeltest': model}, f)
```

Hình 17. Hình ảnh về quá trình rèn luyện mô hình và lưu mô hình

Sau khi rèn luyện và có được mô hình, chúng ta sẽ lưu vào dưới dạng pickle (.p) vì đây là một dạng dữ liệu phức tạp và pickle có thể chứa dữ liệu này thuận tiện cho việc kích hoạt và khởi động lại mô hình khi cần dùng.

3.2.3. Độ chính xác lý thuyết của mô hình

Random Forest là một mô hình học máy mạnh mẽ, nổi bật với khả năng kết hợp dự đoán từ nhiều cây quyết định để cải thiện độ chính xác và giảm thiểu lỗi. Trong quá trình huấn luyện trên tập dữ liệu ASL đã qua xử lý bằng Mediapipe, mô hình đạt độ chính xác lý thuyết 90% trên tập kiểm tra, phản ánh khả năng vượt trội trong việc nhận diện các ký hiệu ngôn ngữ. Các tham số như `n_estimators = 200`, `max_depth = 35`, và `random_state = 42` đóng vai trò quan trọng trong việc đạt được kết quả này, đảm bảo mô hình đủ phức tạp để học các đặc trưng cần thiết nhưng không bị overfitting.

Khi sử dụng *GridViewSearch* để có thể tìm ra độ chính xác lý thuyết của mô hình, ta có thể thấy các đặc tính đầu vào ảnh hưởng ra sao đến độ chính xác của đầu ra. Số liệu được thực thi với dạng ảnh tĩnh và điều kiện lý tưởng nên kết quả sẽ cho ra rất cao.

Model	n_estimators	max_depth	random_state	Mean CV Score
1	100	20	42	99.497%
2	200	20	42	99.525%
3	200	30	42	99.602%
4	200	35	42	99.604%
5	200	40	42	99.602%

Bảng 1. Bảng biểu thị độ chính xác lý thuyết dựa trên các tham số đầu vào

Tuy nhiên, để cải thiện hơn nữa hiệu suất, có thể cân nhắc tăng kích thước tập dữ liệu huấn luyện bằng cách thu thập thêm dữ liệu từ nhiều nguồn hoặc tăng tính đa dạng của dữ liệu đầu vào. Ngoài ra, việc áp dụng các kỹ thuật nâng cao như cross-validation để kiểm tra tính ổn định của mô hình và tinh chỉnh thêm các tham số như n_estimators, max_depth để tìm bộ tham số tối ưu hơn có thể giúp cải thiện kết quả. Bên cạnh đó, tích hợp thêm các đặc trưng bổ sung hoặc sử dụng các mô hình kết hợp khác như Gradient Boosting có thể là một hướng đi tiềm năng để tăng độ chính xác và độ tin cậy.

Như vậy, với nền tảng hiệu suất ban đầu rất tốt, Random Forest không chỉ phù hợp mà còn hứa hẹn đem lại kết quả vượt trội trong việc nhận diện ngôn ngữ ký hiệu ASL khi được cải tiến và tối ưu hóa thêm.

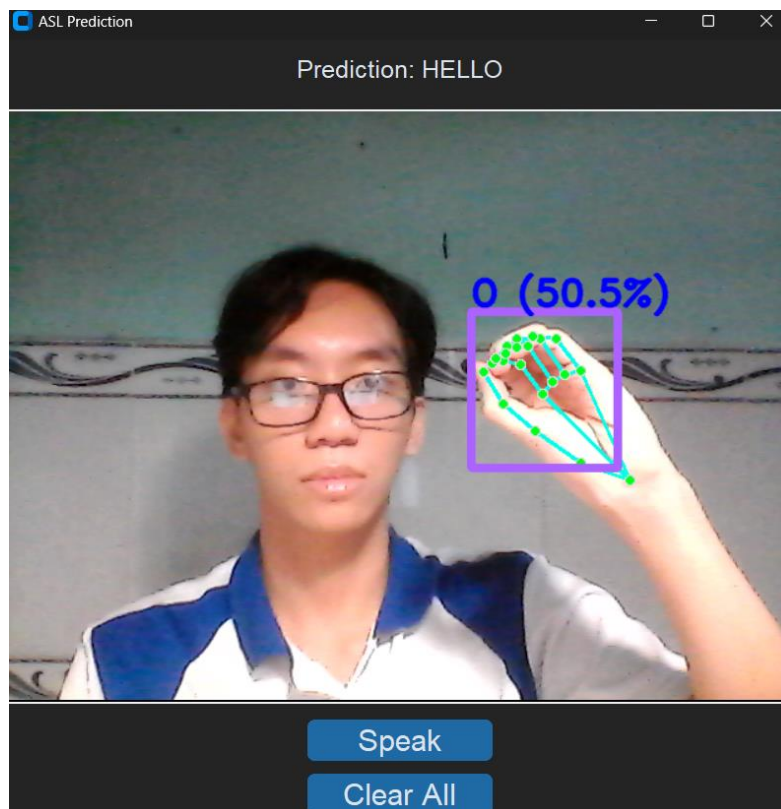
3.3. Các tính năng cải tiến của đề tài

Trong quá trình phát triển ứng dụng nhận diện ngôn ngữ ký hiệu (ASL), chúng tôi đã tích hợp một số tính năng cải tiến nhằm tăng cường trải nghiệm người dùng và khả năng ứng dụng thực tế của hệ thống. Các tính năng này không chỉ giúp người sử dụng nâng cao khả năng giao tiếp mà còn khuyến khích học hỏi và tương tác vui vẻ. Dưới đây là các tính năng nổi bật của hệ thống

3.3.1. Tính năng Text-to-Speech

Tính năng Text-to-Speech (Chuyển văn bản thành giọng nói) là một phần quan trọng trong hệ thống, giúp người sử dụng ngôn ngữ ký hiệu giao tiếp với những người không biết ngôn ngữ ký hiệu. Khi người dùng thực hiện các ký hiệu thông qua camera, hệ thống sẽ nhận diện và chuyển đổi các ký hiệu đó thành văn bản. Sau đó, văn bản sẽ được xử

lý bởi mô-đun *IntoVoice*, một phần của hệ thống, để chuyển đổi văn bản thành giọng nói cũng như xử lý các lỗi trong câu được xuất ra.



Hình 18. Hình ảnh về tính năng đoán ký tự và xuất ra giọng nói

Giới thiệu cụ thể về cách thức hoạt động, khi người dùng thực hiện một ký hiệu cụ thể, hệ thống sẽ hiển thị ký tự tương ứng hoặc một từ trong ngôn ngữ ký hiệu. Nếu người dùng thực hiện đúng và xác nhận ký hiệu là chính xác, hệ thống sẽ xuất ký tự đó ra trên màn hình. Khi người dùng nhấn vào nút “*Speak*”, các ký tự đó sẽ được truyền vào AI để xử lý lỗi và cuối cùng sẽ xuất ra giọng nói thông qua bộ chuyển đổi văn bản thành âm thanh gTTS và trình phát file mp3 của pygame. Nút “*Clear All*” sẽ làm công việc xóa đi hết những nội dung đã dự đoán trên màn hình.

Ngoài ra, các biến như *hold_start_time* và *del_hold_start_time* còn có vai trò quan trọng trong việc ngăn các ký tự được xuất ra liên tục và cũng như xử lý các vấn đề khác như trùng lặp ký tự hay là xóa ký tự.

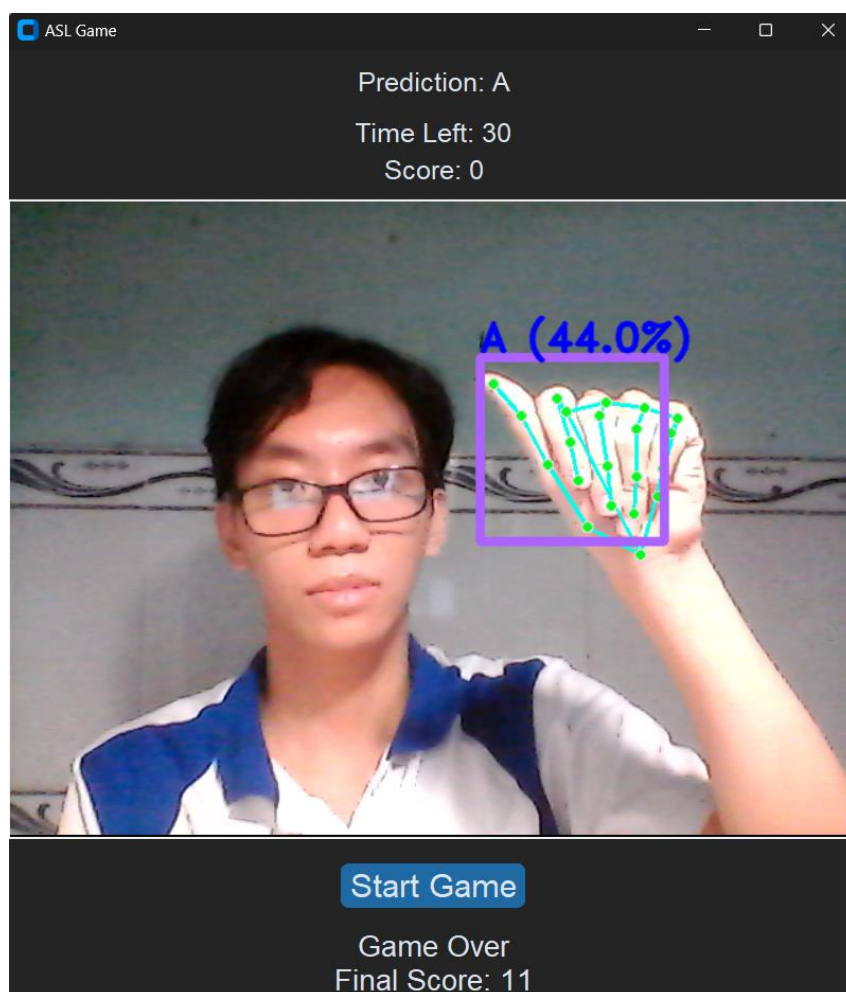
Tính năng dự đoán ký tự này không chỉ hỗ trợ giao tiếp hiệu quả mà còn có ý nghĩa đặc biệt đối với người khiếm thính, giúp họ giao tiếp với cộng đồng không sử dụng ngôn

ngữ ký hiệu. Hệ thống này có thể áp dụng trong nhiều tình huống thực tế, từ việc trò chuyện hàng ngày cho đến việc hỗ trợ trong môi trường học tập hoặc công việc.

3.3.2. Trò chơi với nhận diện ngôn ngữ ký hiệu ASL

Tính năng trò chơi ASL được phát triển để làm tăng tính tương tác và thú vị khi học ngôn ngữ ký hiệu. Trong trò chơi này, người dùng sẽ thực hiện các ký hiệu theo yêu cầu, hệ thống sẽ chấm điểm dựa trên mức độ chính xác của các ký hiệu người chơi thực hiện.

Điểm đặc biệt trong trò chơi này là người chơi có thể tham gia vào phán đoán nhanh các chữ cái trong thời gian chơi. Mỗi lần người dùng thực hiện đúng ký hiệu, điểm số của họ sẽ được tăng lên, và hệ thống sẽ phản hồi bằng cách chơi một hiệu ứng âm thanh vui nhộn, tạo động lực cho người chơi.



Hình 19. Hình ảnh về tính năng trò chơi

Để có thể khiến trò chơi hoạt động một cách trơn tru và hiệu quả, ngoài cách nhận diện ký hiệu từ mô hình máy đã được rèn luyện thì cũng cần phải có các hàm thuận

tiện cho việc kiểm soát trò chơi như bộ đếm thời gian, cập nhật thời gian, hàm bắt đầu, hàm kết thúc trò chơi hay hàm cập nhật điểm số.

```
def start_timer():
    #1st: CHECK IF PLAYER CLICK START AGAIN
    global timeFlag
    if timeFlag == True:
        game_over()
        return

    timeFlag = True #Bc player start, the timeFlag is start -> True
    global time_left
    time_left = 30 # Start from 30 seconds

    #Check if any Label Named : GAME OVER -> DELETE IT
    for widget in game.winfo_children():
        if isinstance(widget, ctk.CTkLabel) and widget.cget("text").startswith("Game Over"):
            widget.destroy()

    #Make the timer start to count down from 30 -> 0
    update_timer()
```

```
def update_timer():
    global time_left, timeFlag
    #Dont update if timeFlag not on yet
    if timeFlag == False:
        return

    #Start to countdown
    if time_left > 0:
        time_left -= 1
        timer_label.configure(text=f"Time Left: {time_left}")
        game.after(1000, update_timer) # Update every second
    else:
        game_over() #Timer reach 0
```

```
def game_over():
    # When over, timeFlag = False -> dont run timer
    # Score back to 0
    # time_left back to 30
    global score, time_left, timeFlag
    timeFlag = False
    game_over_label = ctk.CTkLabel(game, text=f"Game Over\nFinal Score: {score}", font=("Helvetica", 22))
    game_over_label.pack(pady=5)
    score = 0
    time_left = 60
    #Update UI
    score_label.configure(text=f"Score: {score}")
    timer_label.configure(text="Time Left: 30")
    #Make the button work again
    start_button.pack(pady=10)
```

```
def update_score():
    global score
    score += 1
    score_label.configure(text=f"Score: {score}")
```

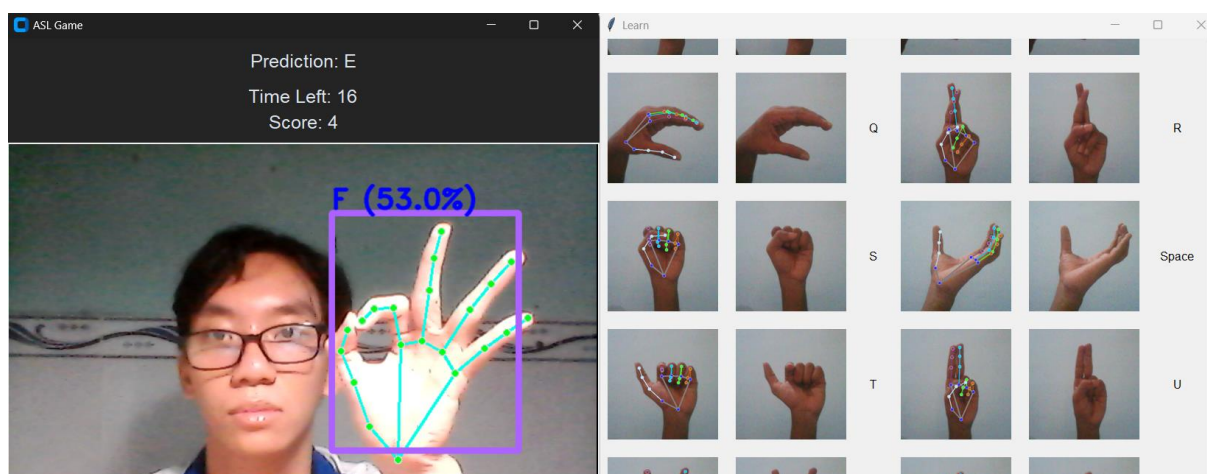
Hình 20. Hình ảnh về các hàm được sử dụng trong trò chơi

Trò chơi này không chỉ là một công cụ học tập mà còn là một phương tiện giải trí sáng tạo, giúp người dùng làm quen với ngôn ngữ ký hiệu một cách thú vị và dễ dàng. Hệ thống này thúc đẩy việc học tập thông qua các nhiệm vụ vui nhộn, vừa học vừa chơi, phù hợp với mọi lứa tuổi và khả năng học của người sử dụng.

3.3.3. Tính năng “Learn the ASL”

Tính năng “Learn the ASL” là tính năng giúp người dùng học ngôn ngữ ký hiệu một cách trực quan và hệ thống. Hệ thống sẽ hiển thị các ký hiệu và cung cấp các hướng dẫn chi tiết về cách thực hiện từng ký hiệu một cách chính xác. Người dùng có thể thực hành theo các chỉ dẫn và nhận phản hồi ngay lập tức về mức độ chính xác của cử chỉ mình thực hiện.

Ngoài ra tính năng này có thể sử dụng song song cùng lúc với hai tính năng trên. Điều này sẽ khiến cho người dùng thuận tiện hơn trong việc thao tác và học tập. Chỉ mất vài phút làm quen là người dùng có thể biết hầu như các ký tự cơ bản trong bảng ngôn ngữ ký hiệu ASL rồi!



Hình 21. Hình tính năng Learn hoạt động song song với trò chơi

Tính năng này là công cụ hữu ích, đặc biệt dành cho những người muốn bắt đầu học ASL một cách bài bản và có hệ thống, giúp họ xây dựng nền tảng vững chắc trong việc giao tiếp bằng ngôn ngữ ký hiệu.

CHƯƠNG 4: KẾT QUẢ THỬ NGHIỆM VÀ ĐÁNH GIÁ TỔNG QUAN

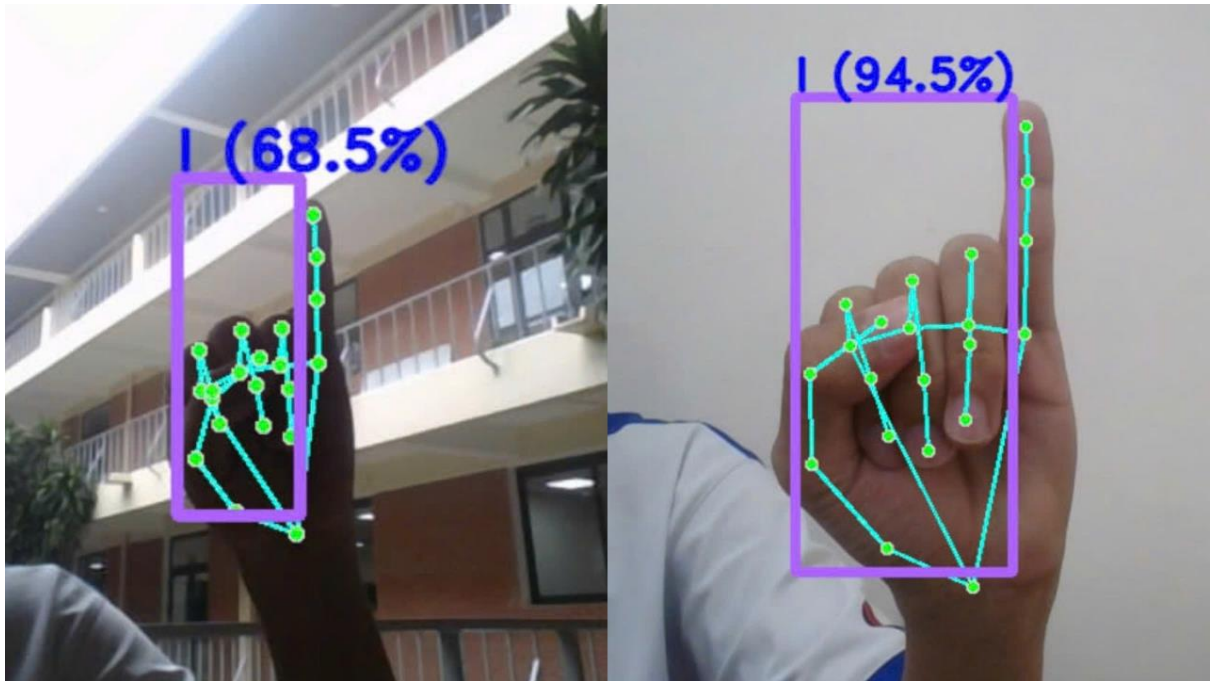
4.1. Kết quả

4.1.1. Độ chính xác thực tế

Độ chính xác của hệ thống được đánh giá thông qua khả năng nhận diện ngôn ngữ ký hiệu từ bàn tay cũng như xử lý văn bản và chuyển đổi chính xác các từ hoặc câu từ đầu vào thành câu hoàn chỉnh với ngữ pháp đúng. Dựa trên đoạn mã đã triển khai, hệ thống sử dụng các mô hình AI như Gemini và Mixtral để tạo ra câu hoàn chỉnh từ các từ viết tắt hoặc không có ngữ pháp.

Kết quả thử nghiệm cho thấy hệ thống có thể đạt được độ chính xác cao trong việc chuyển đổi văn bản đầu vào thành câu hoàn chỉnh. Ví dụ, các đầu vào như *"HELLO-NICE-TO-MEET-U-BUT-I-GTG"* được hệ thống chuyển thành câu *"Hello, nice to meet you, but I have to go."* với ngữ pháp chính xác và dễ hiểu. Tính chính xác của hệ thống được xác nhận qua việc kiểm tra các câu được tạo ra, với phần lớn các câu đều tuân thủ đúng ngữ pháp và ngữ nghĩa.

Về phần trò chơi độ chính xác của hệ thống được đánh giá thông qua khả năng nhận diện cử chỉ tay của người dùng. Hệ thống sử dụng mô hình học máy Random Forest để nhận diện các cử chỉ của bàn tay và dự đoán chữ cái trong ASL (American Sign Language) dựa trên các điểm đặc trưng từ các khớp ngón tay. Các thử nghiệm cho thấy hệ thống có thể nhận diện chính xác chữ cái người dùng ra hiệu trong vòng vài giây và cập nhật điểm số ngay lập tức với độ chính xác dao động từ 60%-90% tùy thuộc vào điều kiện ánh sáng, môi trường xung quanh và độ phân giải của camera.



Hình 22. Hình so sánh độ chính xác giữa môi trường bị ngược sáng và không bị ngược sáng

4.1.2. Kết quả các tính năng

Trong quá trình thử nghiệm, hệ thống cũng đã được kiểm tra qua các tính năng như:

- *Chuyển đổi văn bản thành câu hoàn chỉnh*: Các đoạn văn bản không có ngữ pháp và từ viết tắt được chuyển thành các câu hợp lý, dễ hiểu với ngữ pháp chính xác.

- *Tính năng nhận diện và chuyển văn bản thành giọng nói (TTS)*: Tính năng này hoạt động ổn định, cho phép hệ thống phát âm các câu đã được tạo ra từ văn bản đầu vào. Sử dụng thư viện gTTS, hệ thống có thể chuyển văn bản thành giọng nói với độ rõ ràng và tốc độ phù hợp.

- *Chơi game nhận diện cử chỉ tay (ASL Game)*: Trò chơi yêu cầu người chơi thực hiện các cử chỉ tay tương ứng với chữ cái trong ASL. Người chơi cần đoán chữ cái đúng trong thời gian giới hạn. Khi người chơi đoán đúng, hệ thống phát âm thanh và cập nhật điểm số.

- *Điều khiển âm thanh trong trò chơi*: Hệ thống sử dụng thư viện pygame để phát nhạc nền và âm thanh hiệu ứng khi người chơi đoán đúng chữ cái.

- *Hiển thị thời gian và điểm số*: Trò chơi có đồng hồ đếm ngược, và người chơi phải đoán đúng chữ cái trong thời gian giới hạn để giành điểm.

Các thử nghiệm cho thấy rằng tính năng nhận diện ký tự, TTS (Text-to-Speech) và cả trò chơi đều hoạt động mượt mà và đáp ứng được yêu cầu của người dùng về việc nghe lại văn bản được tạo ra từ hệ thống.

4.2. Đánh giá tổng quan

Sau khi thử nghiệm hệ thống với nhiều đầu vào khác nhau, nhận thấy rằng hệ thống có thể nhận diện ký tự và xử lý các văn bản phức tạp khá tốt, đặc biệt là các câu sử dụng từ viết tắt và không đầy đủ ngữ pháp. Hệ thống xử lý tốt trong việc tạo ra các câu hợp lý và chính xác, đồng thời có khả năng chuyển đổi thành giọng nói rõ ràng, dễ nghe.

Các mô hình AI được tích hợp (Gemini và Mixtral) đã chứng tỏ được hiệu quả trong việc tạo ra các câu văn phù hợp, mặc dù một số trường hợp cần chuyển sang mô hình Mixtral nếu mô hình Gemini gặp phải các hạn chế nhất định. Tuy nhiên, tổng thể, cả hai mô hình đều hoạt động hiệu quả và đáp ứng yêu cầu của người dùng.

Về mặt hiệu năng, hệ thống hoạt động ổn định và nhanh chóng, với độ trễ thấp khi xử lý văn bản và chuyển đổi thành giọng nói. Các thử nghiệm cho thấy hệ thống có thể xử lý nhiều đầu vào mà không gặp phải lỗi hoặc sự cố.

Trò chơi hoạt động một cách trơn tru và hiệu quả, các cơ chế hoạt động cũng như âm thanh được thử nghiệm một cách chính xác. Bộ đếm thời gian và cập nhật điểm số được thực hiện mỗi lần người dùng bắt đầu và đạt được điểm số.

Hệ thống học được thể hiện một cách trực quan và dễ nhìn giúp cho người dùng có thể thao tác và học tập một cách dễ dàng. Phần học hoạt động song song với hai phần chính để người dùng có một trải nghiệm tốt nhất.

Tổng quan, kiến trúc hệ thống từ menu liên kết đến với các cơ chế khác không gặp lỗi và không bị gặp các vấn đề về thao tác hay giao diện.

KẾT LUẬN VÀ KHUYẾN NGHỊ

1. Kết luận

Trong bối cảnh công nghệ ngày càng phát triển và ứng dụng vào nhiều lĩnh vực đời sống, việc nghiên cứu và phát triển hệ thống nhận diện ngôn ngữ ký hiệu Mỹ (ASL) đóng một vai trò quan trọng và mang lại nhiều giá trị thiết thực. Đề tài này không chỉ thể hiện sự tiến bộ vượt bậc trong lĩnh vực trí tuệ nhân tạo và học máy mà còn mở ra những cơ hội ứng dụng rộng rãi trong thực tiễn, góp phần cải thiện cuộc sống của người khiếm thính và thúc đẩy sự hòa nhập xã hội.

Qua quá trình nghiên cứu và triển khai dự án, các công nghệ hiện đại như xử lý ảnh, học máy, và nhận diện hình ảnh đã được áp dụng một cách hiệu quả để phát triển một hệ thống có khả năng nhận diện chính xác các cử chỉ ngôn ngữ ký hiệu. Hệ thống này không chỉ đáp ứng các yêu cầu kỹ thuật mà còn đảm bảo tính thân thiện và dễ sử dụng, tạo điều kiện thuận lợi cho người dùng tiếp cận và sử dụng.

Một trong những lợi ích lớn nhất của hệ thống nhận diện ASL là khả năng hỗ trợ giao tiếp giữa người khiếm thính và những người không biết ngôn ngữ ký hiệu. Điều này không chỉ giúp cải thiện chất lượng cuộc sống của người khiếm thính mà còn góp phần thúc đẩy sự hiểu biết và thông cảm lẫn nhau trong cộng đồng. Trong môi trường giáo dục, y tế và các dịch vụ, hệ thống này có thể được ứng dụng để nâng cao hiệu quả giao tiếp, tạo điều kiện thuận lợi cho mọi người tiếp cận thông tin và dịch vụ.

Ngoài ra, việc phát triển hệ thống này còn mang ý nghĩa nhân văn sâu sắc, thể hiện sự quan tâm và tôn trọng đối với quyền lợi của tất cả các thành viên trong xã hội. Qua đó, đề tài này góp phần nâng cao nhận thức của cộng đồng về những khó khăn mà người khiếm thính phải đối mặt, từ đó thúc đẩy sự đồng cảm và hỗ trợ lẫn nhau.

Việc ứng dụng các thư viện và công cụ như OpenCV, Pandas, Pickle, và Tkinter trong quá trình phát triển hệ thống đã chứng minh tính hiệu quả và linh hoạt của chúng trong việc xử lý các bài toán phức tạp và xây dựng các ứng dụng thực tiễn. Những công cụ này không chỉ giúp đơn giản hóa quá trình phát triển mà còn nâng cao hiệu suất và tính ổn định của hệ thống, đảm bảo đáp ứng tốt các yêu cầu của người dùng cuối.

Tóm lại, dự án nhận diện ngôn ngữ ký hiệu Mỹ mang lại nhiều giá trị vượt trội từ khoa học công nghệ đến xã hội và nhân văn. Hệ thống không chỉ góp phần thúc đẩy tiến bộ công nghệ mà còn cải thiện chất lượng cuộc sống của người khiếm thính, tạo ra một xã hội toàn diện và hòa nhập hơn. Đây là một đề tài xứng đáng được đầu tư nghiên cứu và phát triển, vì những giá trị to lớn mà nó mang lại cho cả cá nhân và cộng đồng. Chúng em tin rằng với sự tiếp tục nghiên cứu và cải tiến, hệ thống này sẽ ngày càng hoàn thiện hơn, đáp ứng được những yêu cầu ngày càng cao của xã hội và mang lại nhiều lợi ích thiết thực.

2. Hướng phát triển trong tương lai

Để hệ thống nhận diện ngôn ngữ ký hiệu Mỹ (ASL) ngày càng hoàn thiện và hữu ích hơn, các hướng phát triển trong tương lai có thể bao gồm:

Nâng cao độ chính xác và hiệu suất:

- Cải tiến thuật toán: Tìm kiếm và áp dụng các thuật toán học máy tiên tiến hơn để nâng cao độ chính xác của hệ thống. Có thể xem xét sử dụng các mô hình học sâu (deep learning) như CNNs (Convolutional Neural Networks) hoặc RNNs (Recurrent Neural Networks) để xử lý và nhận diện ngôn ngữ ký hiệu một cách hiệu quả hơn. Ngoài ra còn có thể sử dụng các model mới đang phát triển như Yolo để có thể nhận diện nhiều tay trên một khung hình (frame).

- Mở rộng tập dữ liệu huấn luyện: Thu thập thêm dữ liệu từ nhiều nguồn khác nhau để đảm bảo tính đa dạng và phong phú, giúp mô hình học máy học được từ nhiều trường hợp và tình huống khác nhau. Điều này sẽ giúp hệ thống có khả năng nhận diện chính xác hơn trong các điều kiện thực tế đa dạng.

Tích hợp công nghệ mới:

- Nhận diện cử chỉ động: Phát triển hệ thống có khả năng nhận diện cử chỉ ngôn ngữ ký hiệu không chỉ từ ảnh tĩnh mà còn từ video, giúp tăng cường khả năng nhận diện trong các tình huống giao tiếp thực tế.

- Công nghệ thực tế ảo (VR) và thực tế tăng cường (AR): Tích hợp hệ thống với công nghệ VR và AR để tạo ra môi trường học tập và giao tiếp tương tác hơn. Ví dụ, chế tạo một cánh tay robot phù hợp với người dùng, từ đó người dùng có thể học và giao tiếp ngôn ngữ ký hiệu trong thời gian thực. Điều này sẽ rất hữu ích khi cần trò chuyện nhưng người khác lại không thành thạo về ngôn ngữ ký hiệu.

Tăng cường khả năng tương tác và thân thiện với người dùng:

- Cải tiến giao diện người dùng (UI/UX): Tiếp tục nâng cao giao diện và trải nghiệm người dùng, đảm bảo hệ thống dễ sử dụng và thân thiện với mọi đối tượng người dùng, bao gồm cả những người không rành về công nghệ.

- Đa ngôn ngữ và đa nền tảng: Phát triển phiên bản hệ thống hỗ trợ nhiều ngôn ngữ và có thể chạy trên nhiều nền tảng khác nhau, từ máy tính, điện thoại di động đến các thiết bị đeo tay thông minh.

Nghiên cứu và phát triển cộng đồng:

- Hợp tác nghiên cứu: Hợp tác với các tổ chức, viện nghiên cứu và các chuyên gia trong lĩnh vực học máy, xử lý ảnh và ngôn ngữ ký hiệu để tiếp tục nghiên cứu và cải tiến hệ thống.

- Cộng đồng nguồn mở: Tham gia và đóng góp vào các dự án nguồn mở như github hay kaggle để phát triển cộng đồng, chia sẻ kiến thức và tài nguyên, cùng nhau nâng cao hệ thống nhận diện ngôn ngữ ký hiệu.

TÀI LIỆU THAM KHẢO

1. Al Fath Terry (4/2024), American Sign Language real-time detection, <https://www.kaggle.com/code/alfatherry/american-sign-language-real-time-detection/notebook>, 03/12/2024.
2. Abdullah Mujahid (2020), American-Sign-Langugae, <https://github.com/abdullahmujahidali/American-Sign-Language?tab=readme-ov-file>, 03/12/2024.
3. Sovit Ranjan Rath (2020), American Sign Language Detection using Deep Learning, <https://github.com/sovit-123/American-Sign-Language-Detection-using-Deep-Learning>, 03/12/2024.
4. Computer vision engineer (26/01/2023), sign-language-detector-python, <https://github.com/computervisioneng/sign-language-detector-python>, 03/12/2024.
5. Google AI for Developers, MediaPipe Solution Guide <https://ai.google.dev/edge/mediapipe/solutions/guide>, 03/12/2024.
6. Scikit-learn, User Guide: Ensembles: Gradient boosting, random forests, bagging, voting, stacking, <https://scikit-learn.org/stable/modules/ensemble.html>, 03/12/2024.